

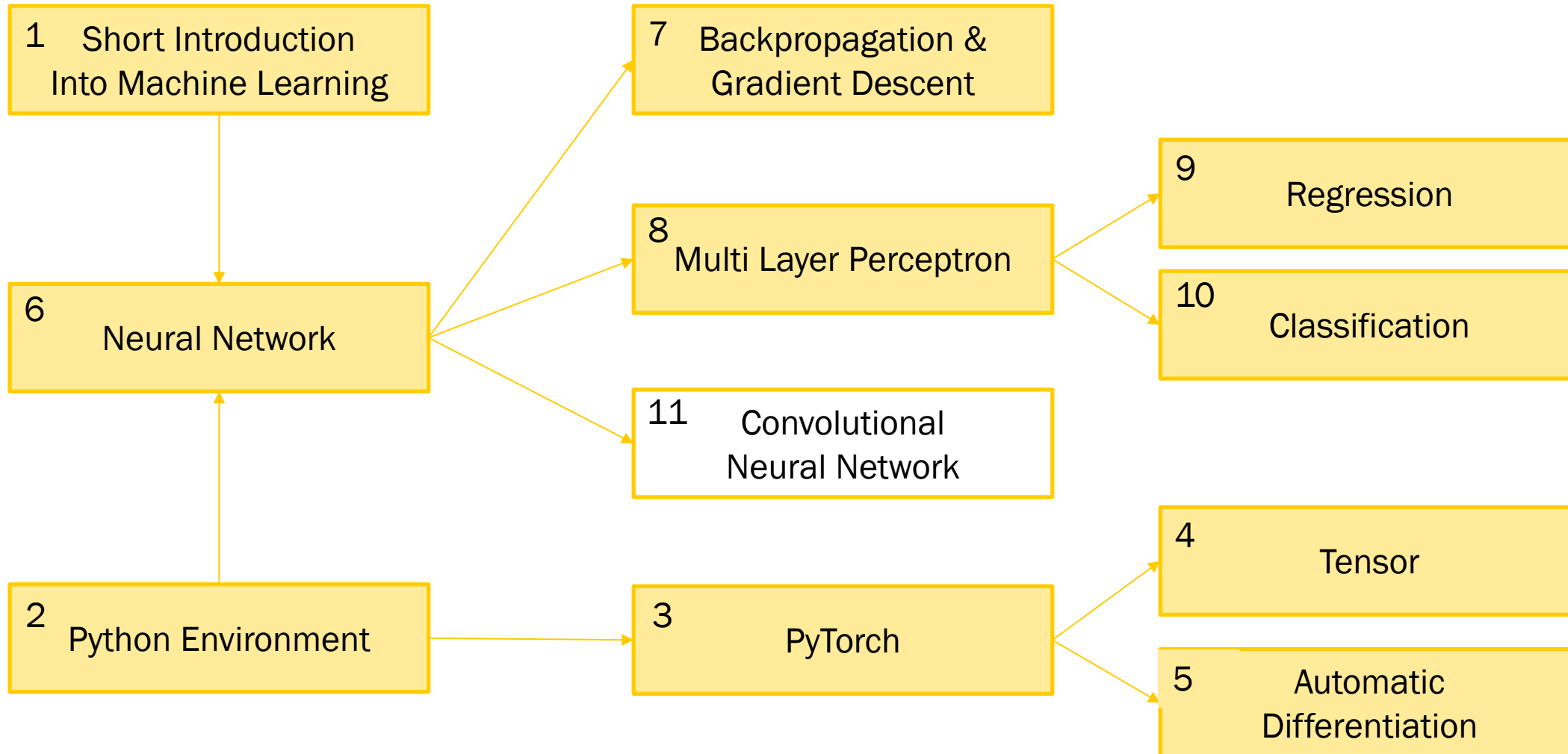
# Intelligente Informationssysteme

## 3 – Agentic AI

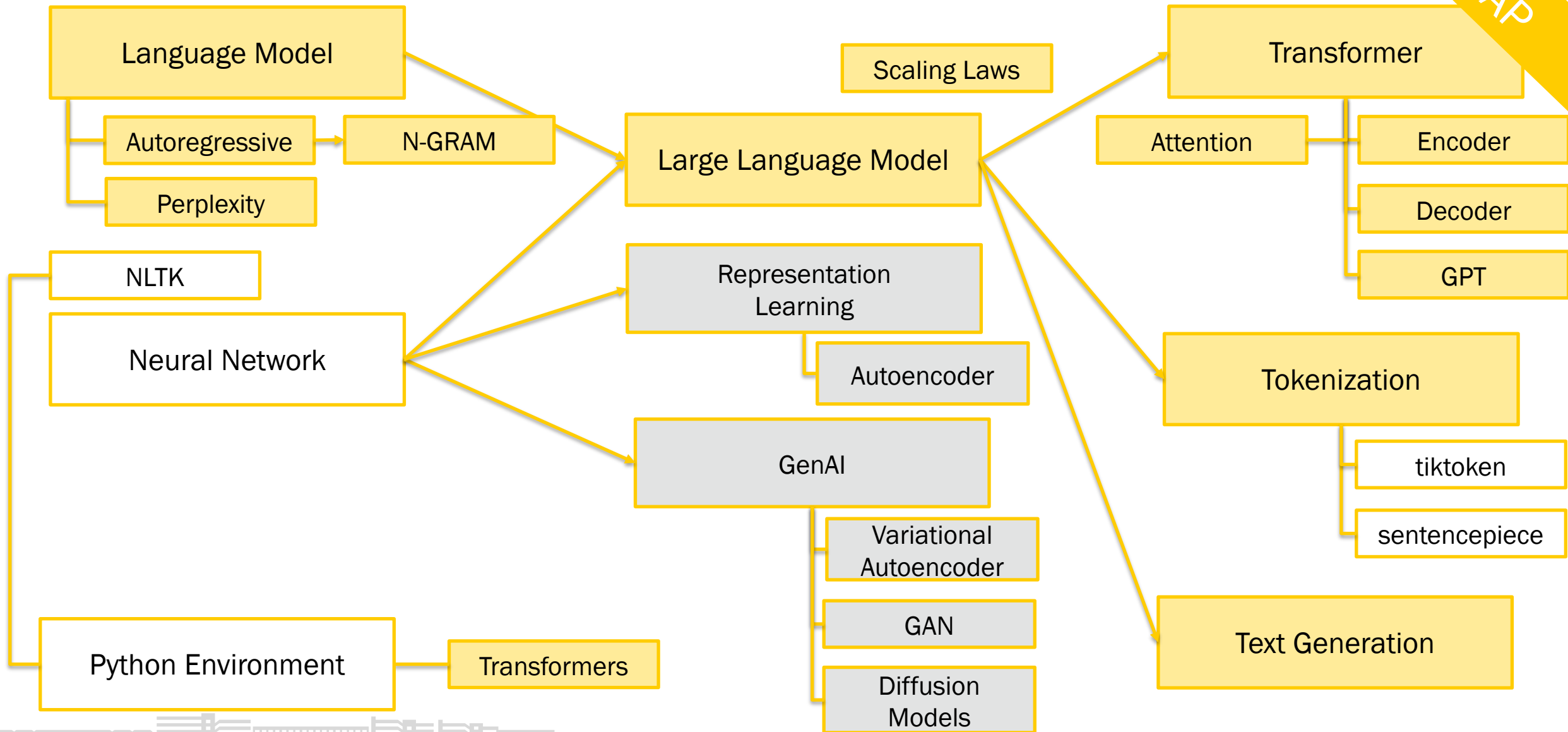
Dominik Neumann



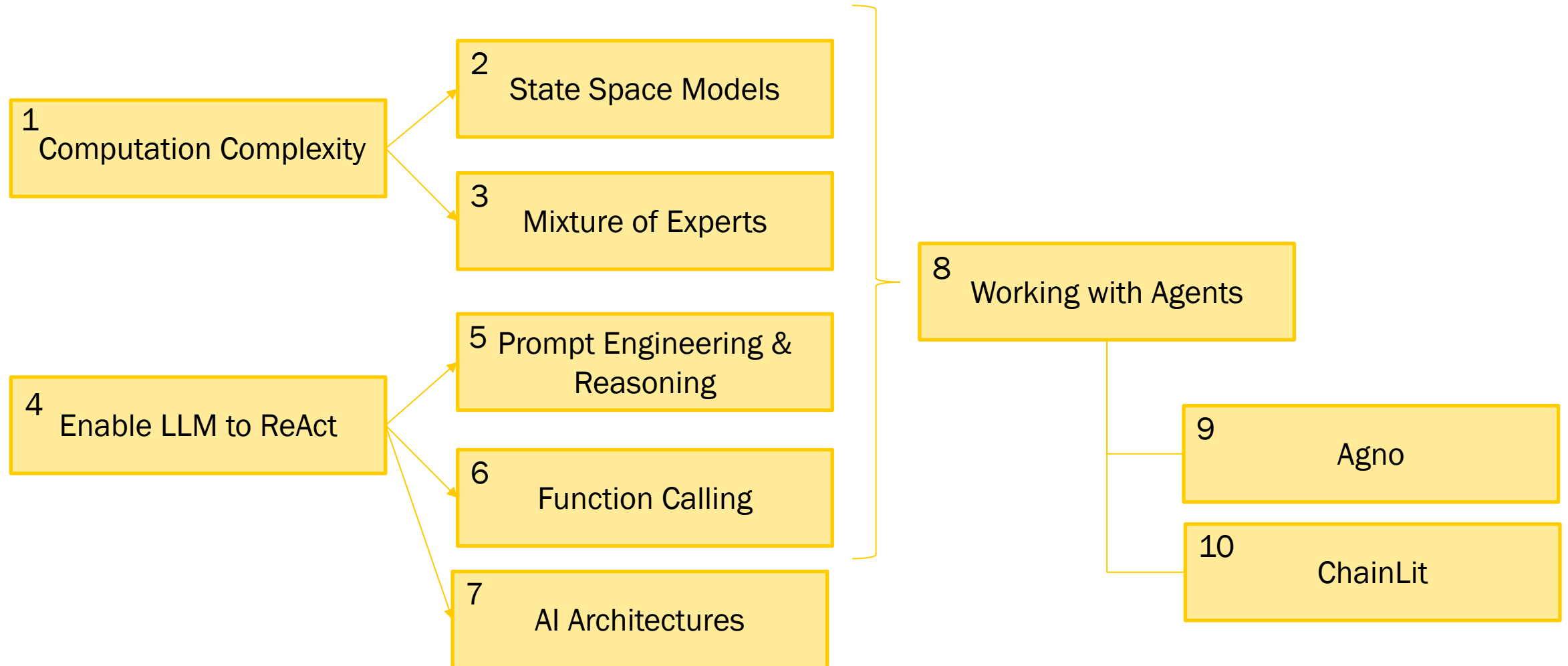
# Block 1 – Neural Network



# Block 2 – Generative AI & Transformer

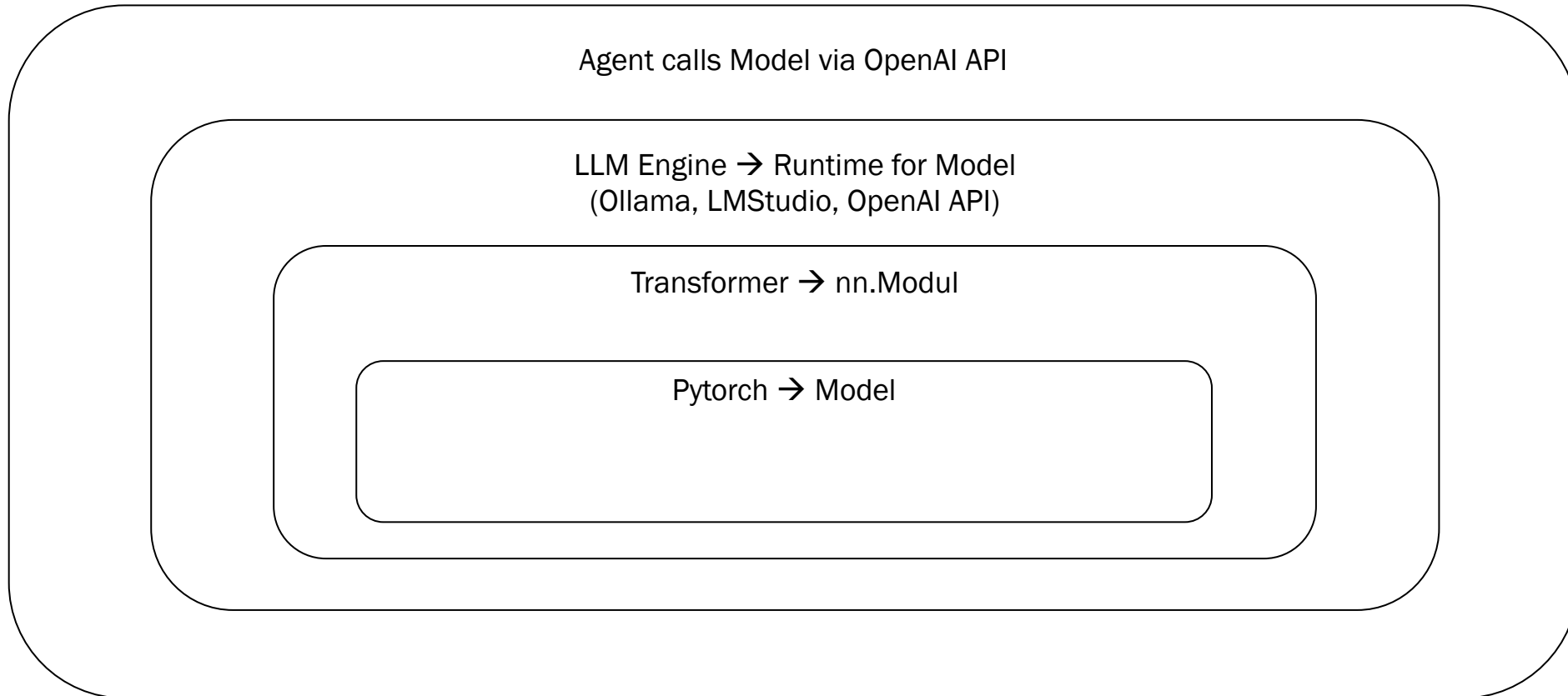


# Bock 3 – Agentic AI





# Layers of Abstraction

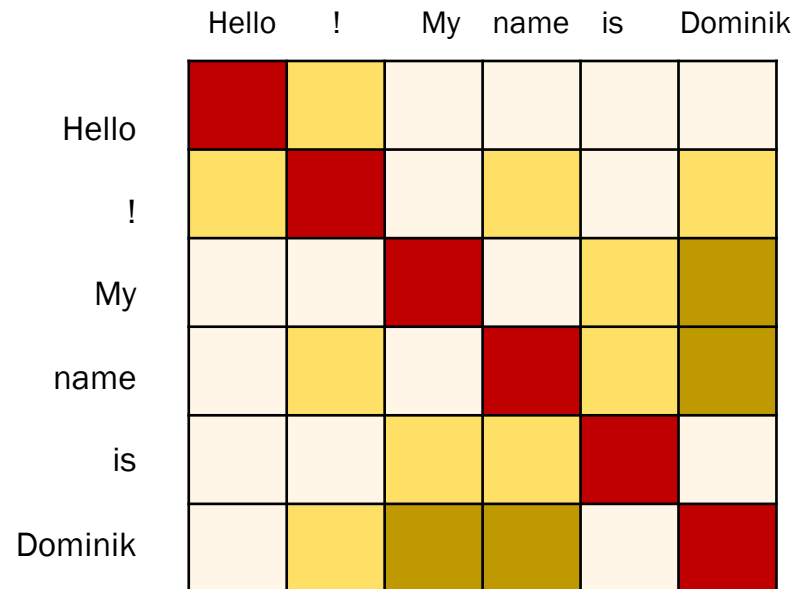
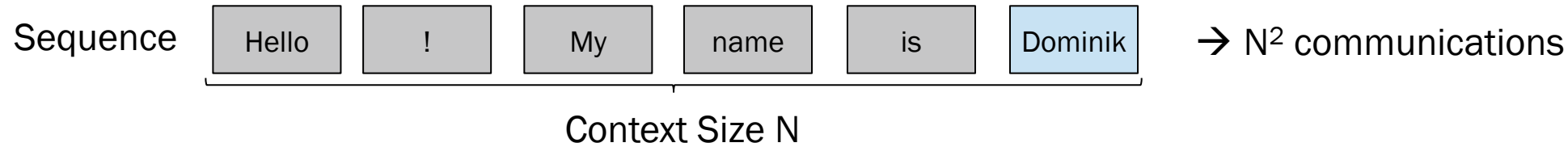


# 03.01 Computational Complexity



# Computational Complexity

What is the disadvantage of Transformers?



This pairwise communication means a forward pass is

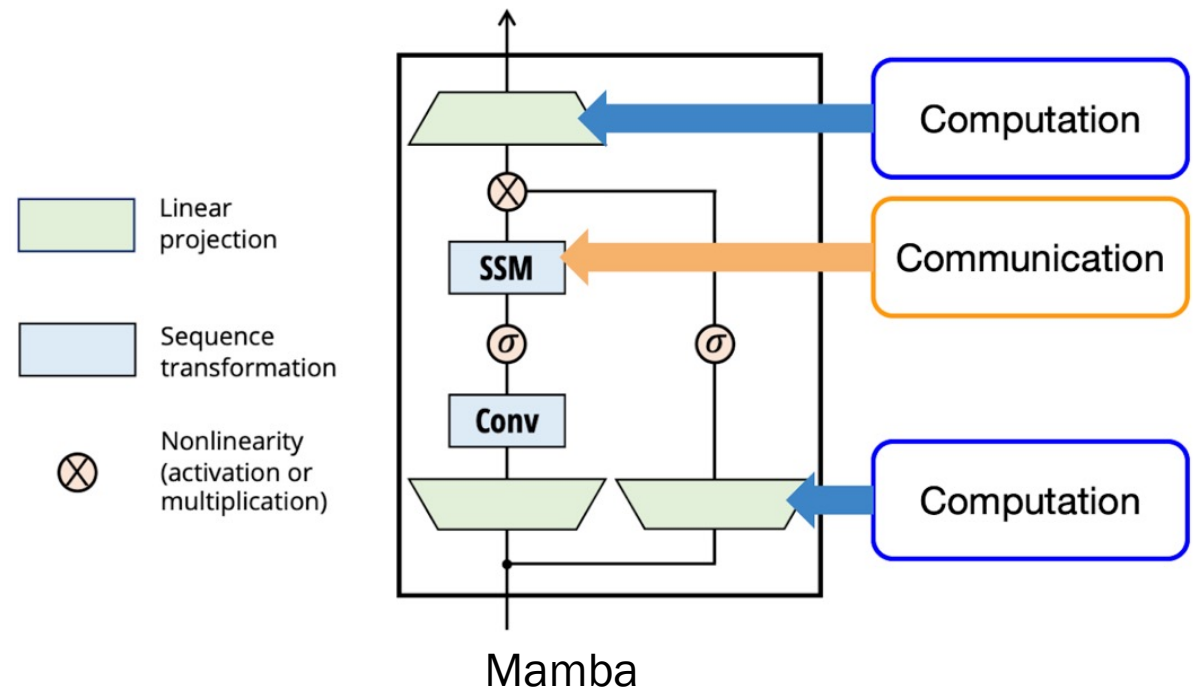
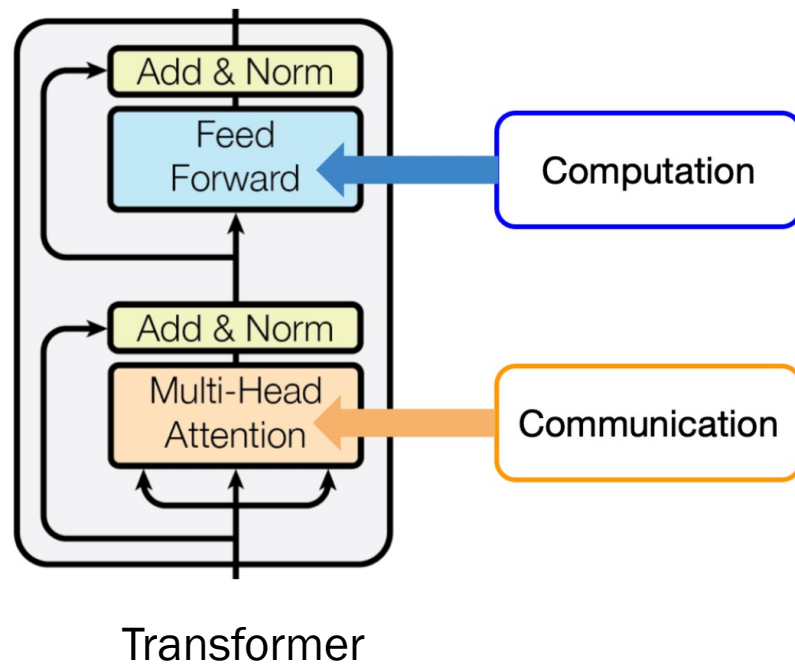
- $O(N^2)$  time complexity in training
- and each new token generated autoregressively takes  $O(N)$  time.

In other words, as the context size increases, the model gets slower.

# Transformer & Mamba compared

Foundation Model Backbones have components for two important operations:

- Communication between tokens
- Computation within a token

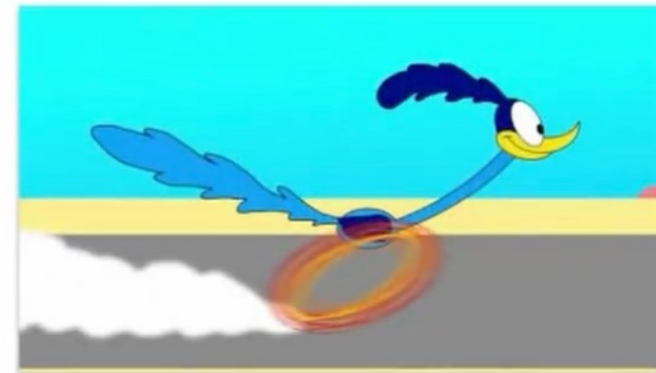


## 03.02 State Space Models



Foundation models are built for batch workloads that require **slow intelligence**.

Interactive applications require **real-time intelligence**.



**Real-time intelligence** will transform  
a wide range of interactive applications.



Conversational  
Interfaces



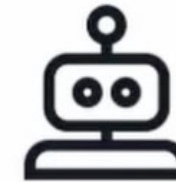
Always-on  
Assistants



World Generation  
in Games & VR



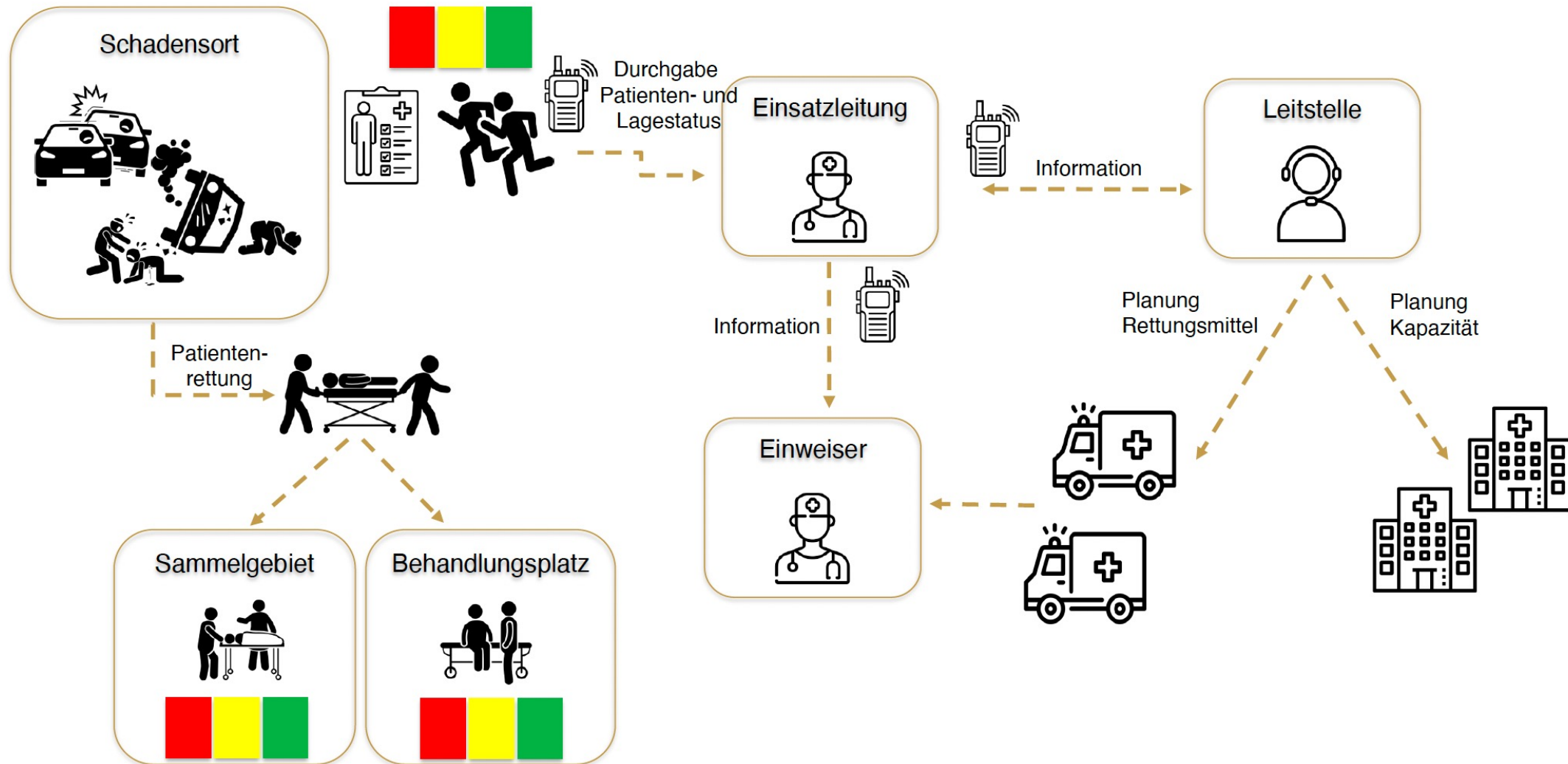
AI Across  
Devices



Robotics

Many of these applications will run on-device.

## Example





## Definition:

A state-space model is a mathematical framework that uses a set of first-order differential or difference equations to describe the behavior of a system.

It represents a system's dynamics by tracking its internal "state variables," which are not always directly observable, and how they are affected by inputs to produce outputs.

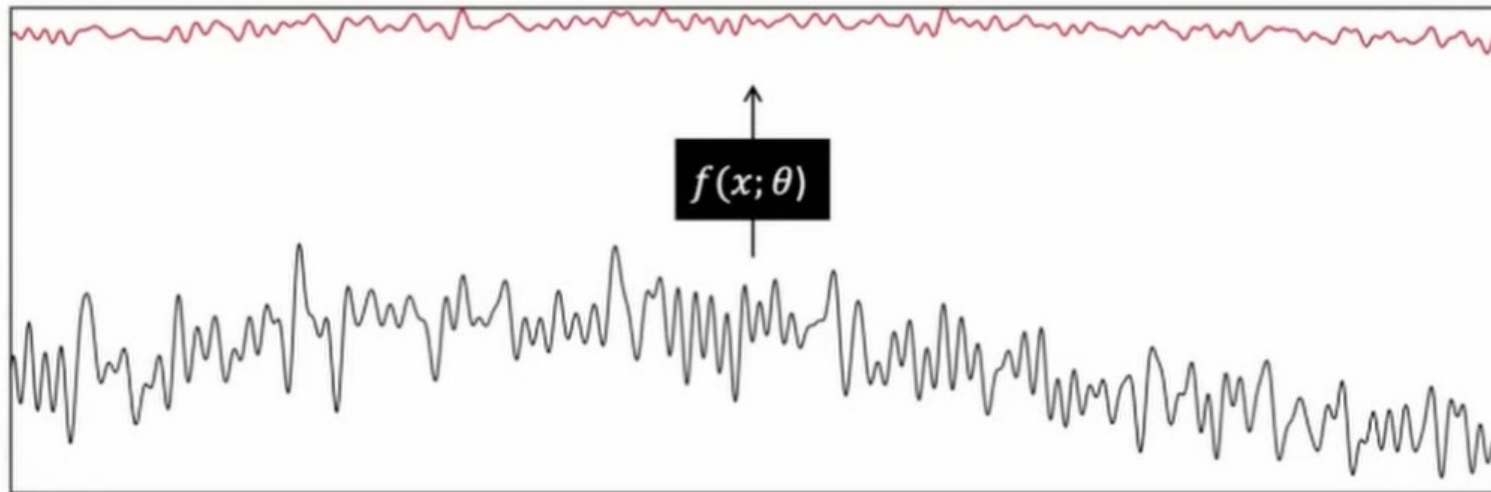
**State Equation:** This equation describes how the internal state of the system changes over time in response to the current state and the system's inputs. For a continuous-time system:

$$h'(t) = A * h(t) + B * x(t)$$

**Output Equation:** This equation describes the system's observable output based on its current state and inputs:

$$y(t) = C * h(t) + D * x(t)$$

# Example

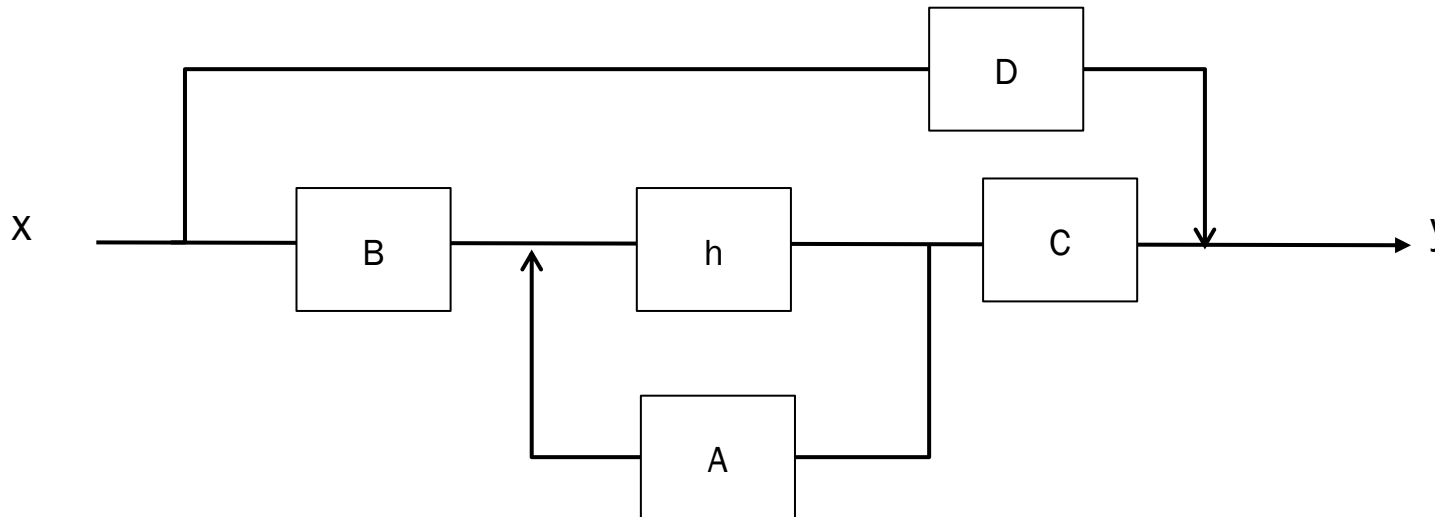


SSMs map 1D **function** to 1D **function**

The Equations:

$$h'(t) = A * h(t) + B * x(t)$$

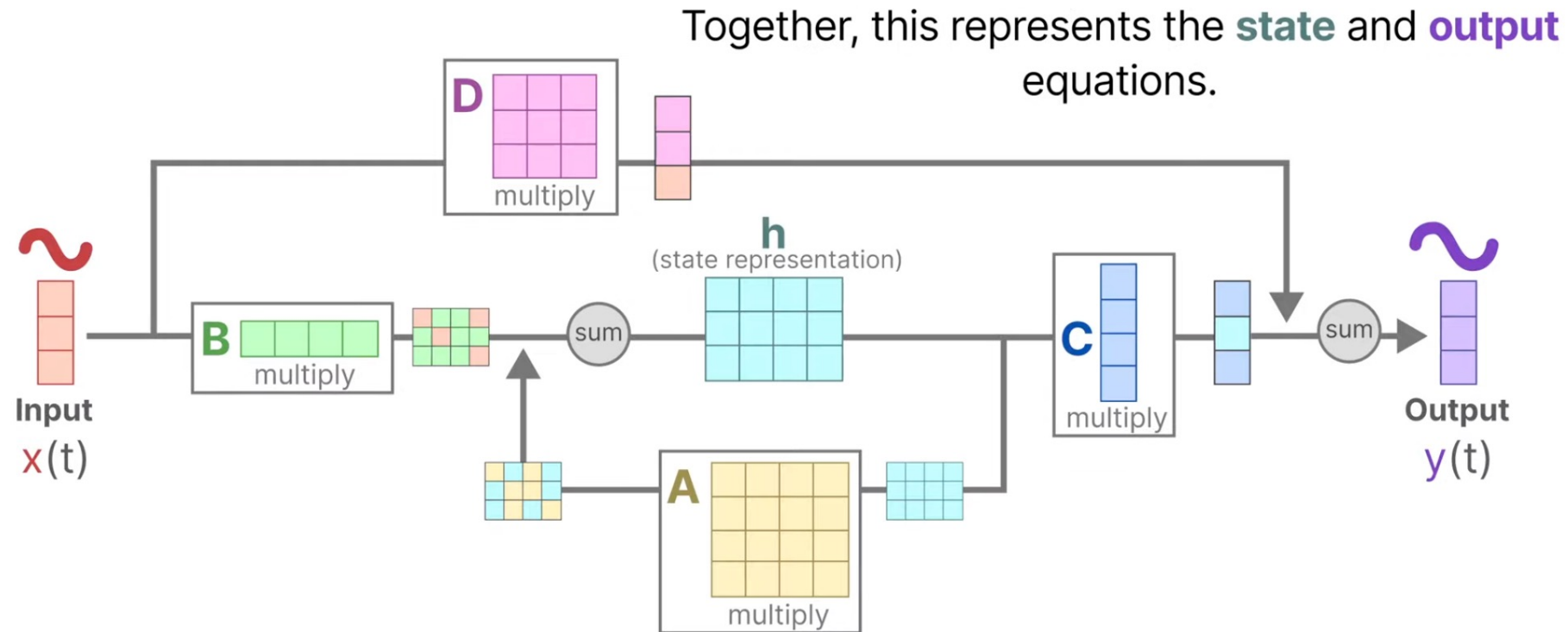
$$y(t) = C * h(t) + D * x(t)$$



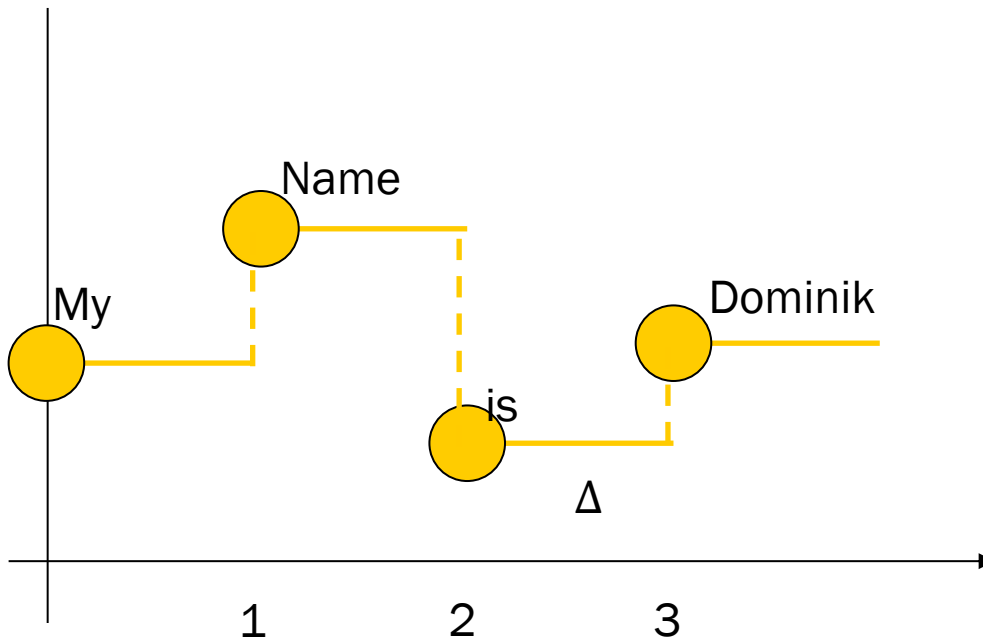
Key Components:

- $h(t)$  the system's state vector
- $x(t)$  the input vector
- $y(t)$  the output vector
- $A$  the state matrix
- $B$  the input matrix
- $C$  the output matrix
- $D$  the transmission matrix

# State Space Model



How can a discrete text signal be processed by a continuous model?



1. Assume we have a discrete input signal like text
2. We hold each signals value until the next signal arrives
3. This creates an artificial continuous signal
4. Space between signals is step-size  $\Delta$  (learned parameter)
5. This process discretizes matrices A and B using learned step size:

$$\bar{A} = \exp(\Delta A)$$

$$\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \Delta B$$

## The Recurrent Representation

This **linear sequence of steps** might remind you of something similar!

Timestep 0

$$\begin{aligned}h_0 &= \bar{B}x_0 \\ y_0 &= Ch_0\end{aligned}$$

Timestep 1

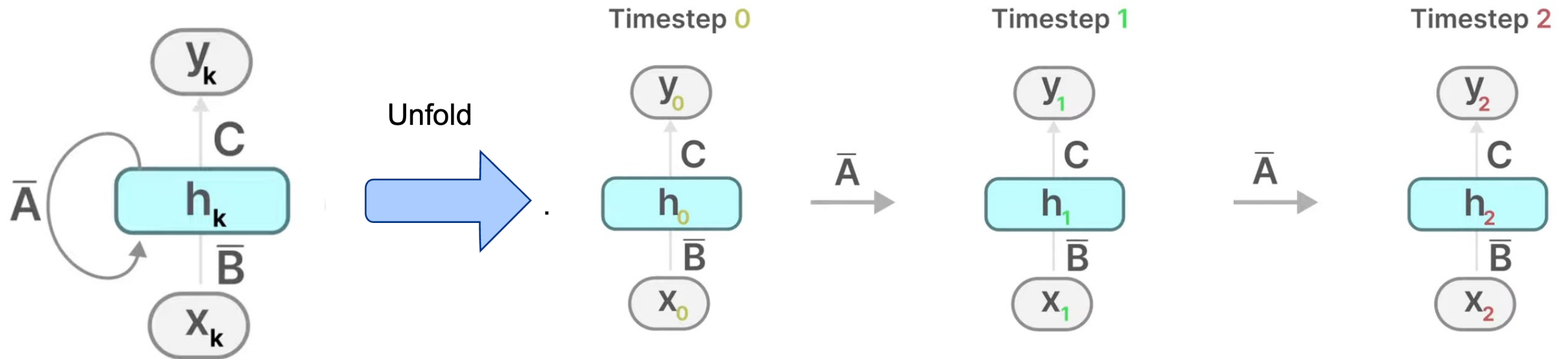
$$\begin{aligned}h_1 &= Ah_0 + Bx_1 \\ y_1 &= Ch_1\end{aligned}$$

Timestep 2

$$\begin{aligned}h_2 &= Ah_1 + Bx_2 \\ y_2 &= Ch_2\end{aligned}$$

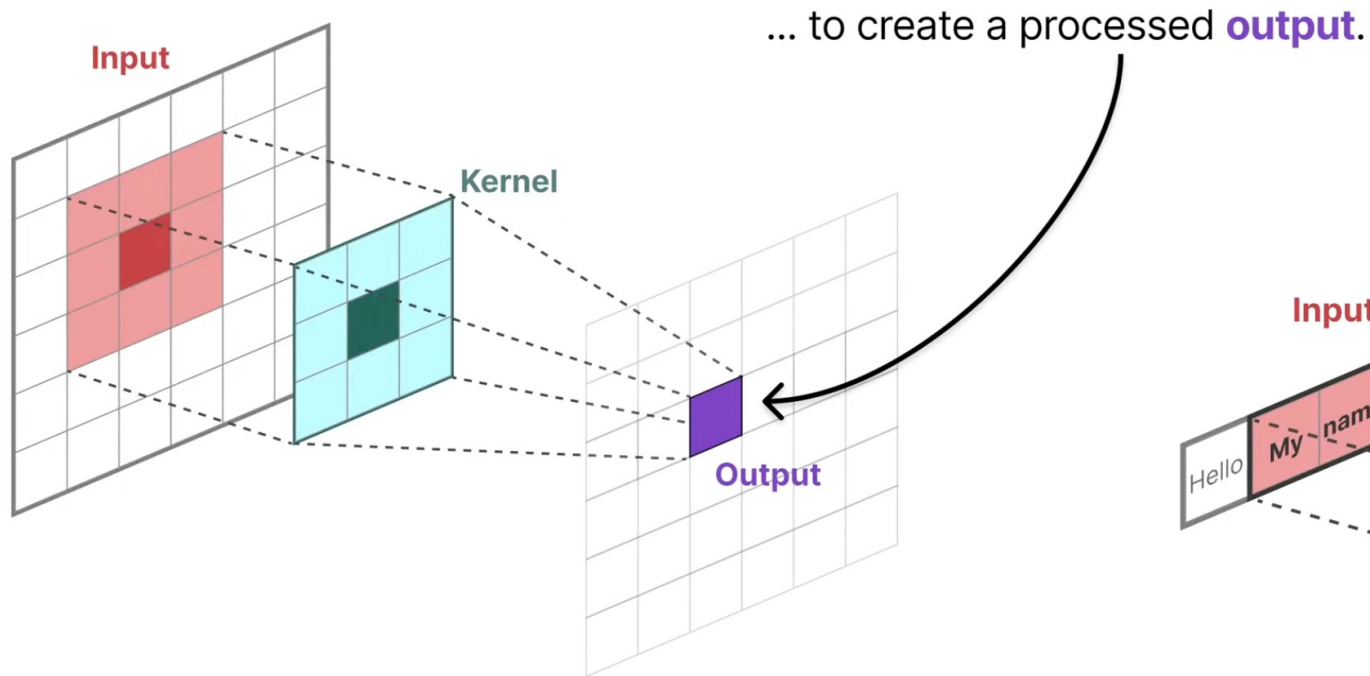
# State Space Model

State Space Models can be represented as RNNs

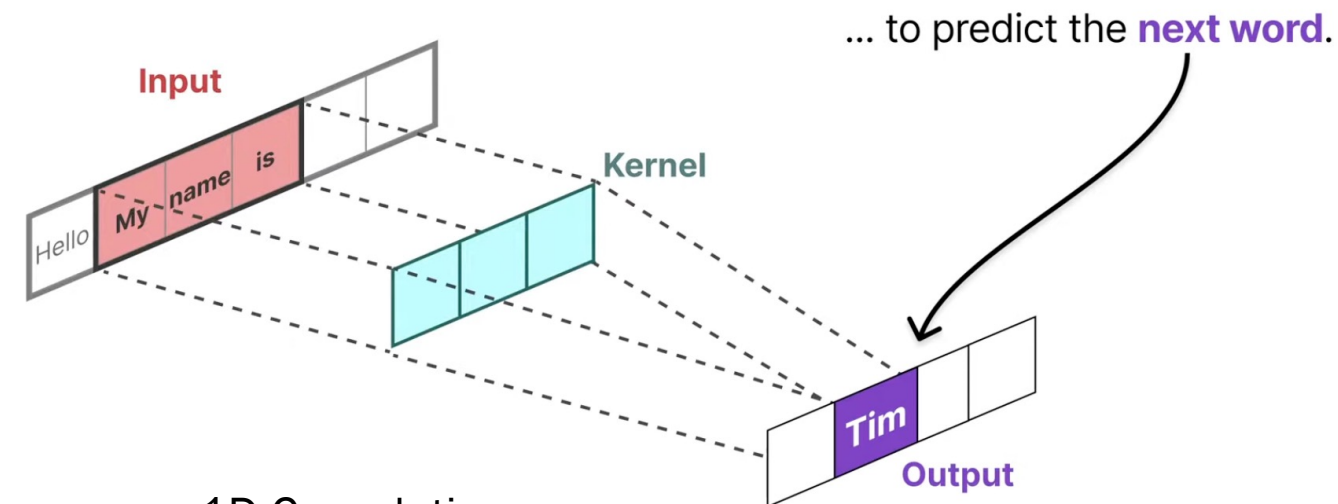


# State Space Models

State Space Models can be represented as CNN



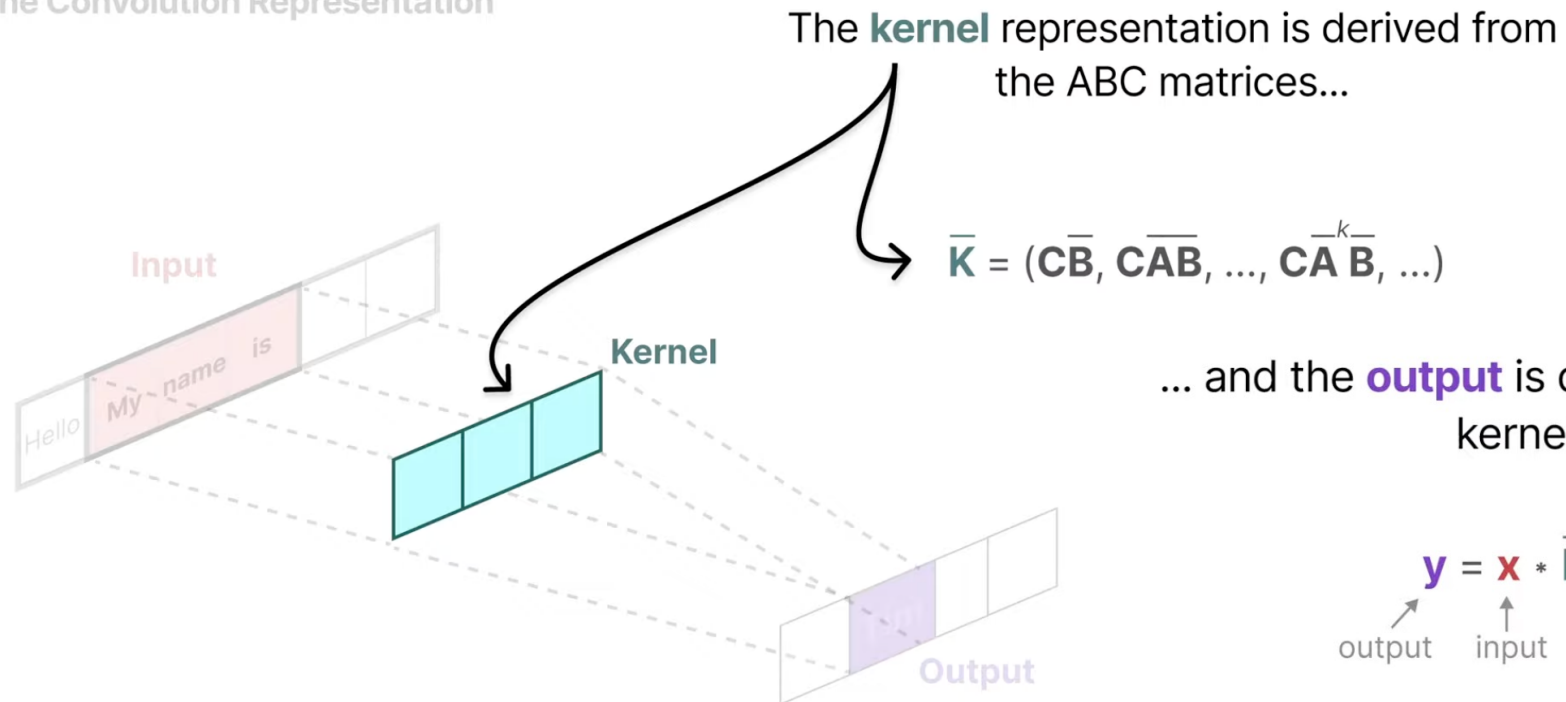
2D-Convolution



1D-Convolution



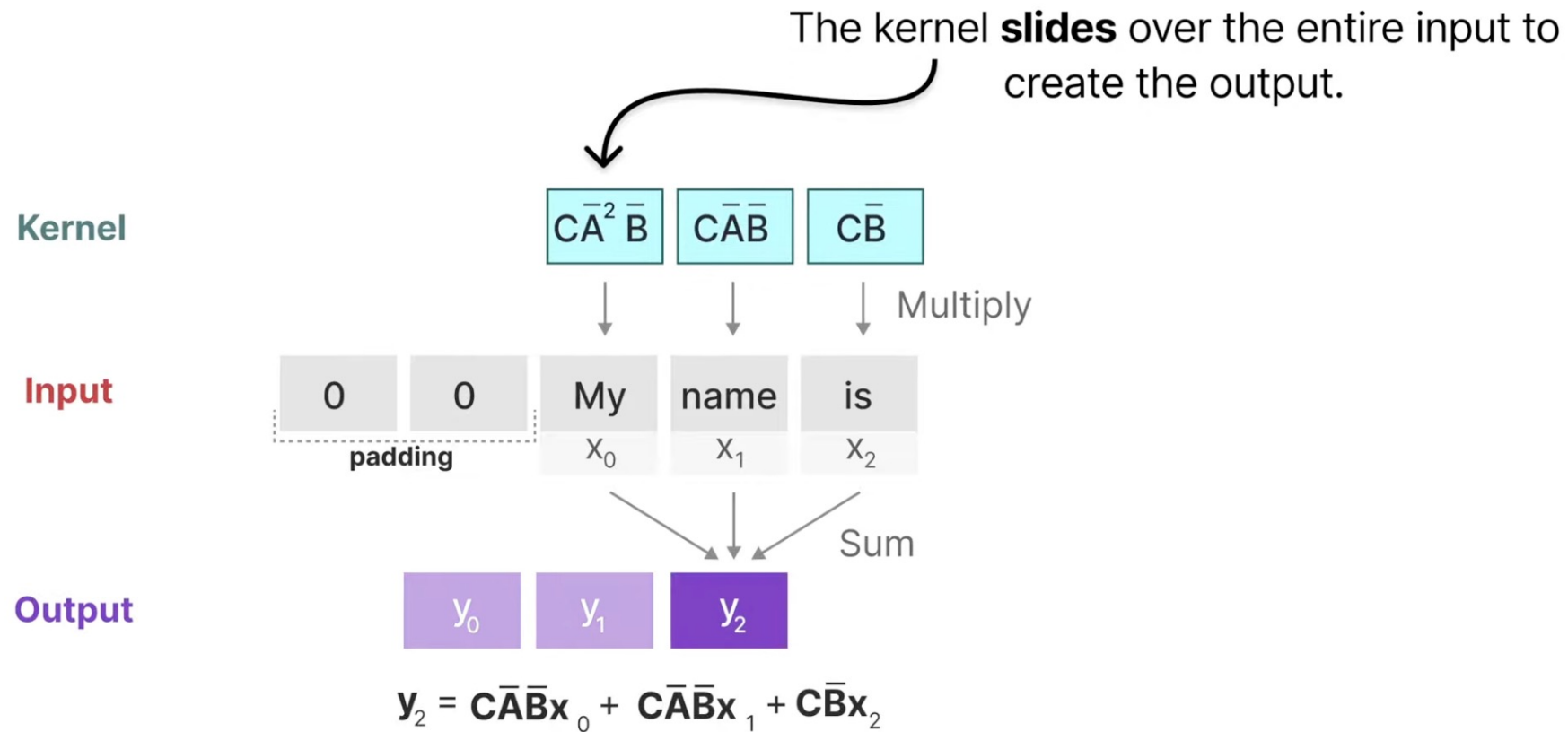
## The Convolution Representation



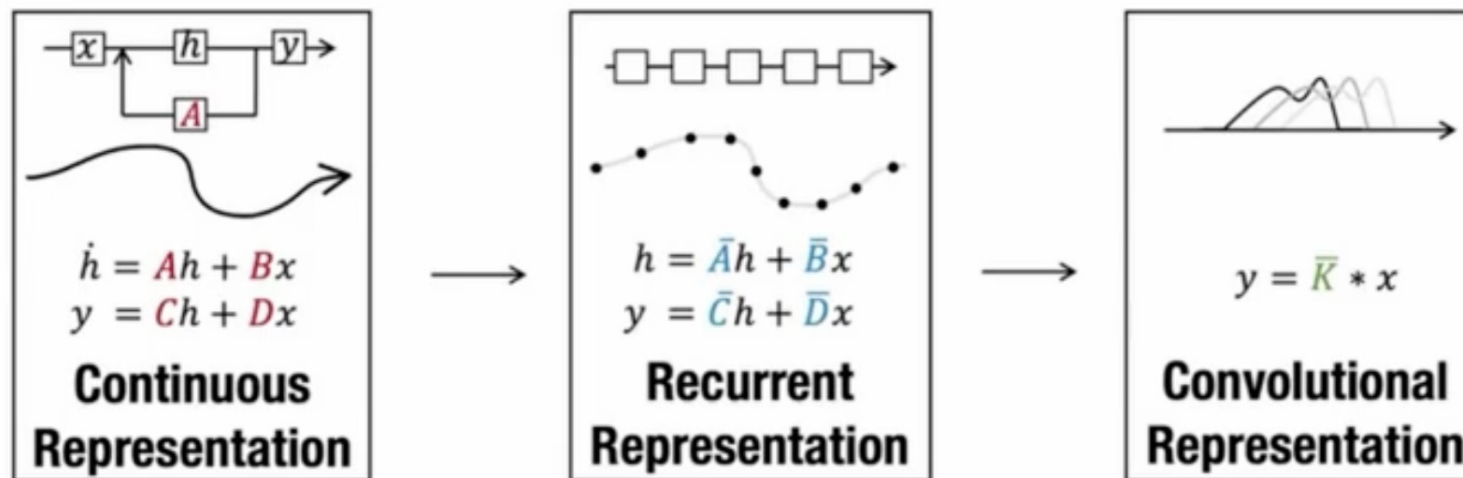
$$\mathbf{y} = \mathbf{x} * \bar{\mathbf{K}}$$

output    input    kernel

## The Convolution Representation



## Structured State Space Models (S4)



**Deep learning** model related to **SSMs**, **RNNs**, **CNNs**

# State Space Models

## S4 – Paper 2022

## S5 – Paper 2023

## S6 (Mamba) 2024

## Jamba 2024

### Efficiently Modeling Long Sequences with Structured State Spaces

Albert Gu, Karan Goel, and Christopher Ré  
Department of Computer Science, Stanford University  
{albertgu,krng}@stanford.edu, chrisre@cs.stanford.edu

#### Abstract

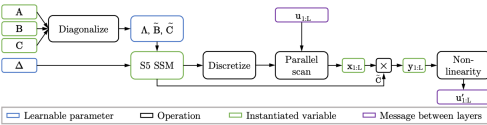
A central goal of sequence modeling is designing a single principled model that can address sequence data across a range of modalities and tasks, particularly on long-range dependencies. Although conventional models including RNNs, CNNs, and Transformers have specialized variants for capturing long dependencies, they still struggle to scale to very long sequences of 10000 or more steps. A promising recent approach proposed modeling sequences by simulating the fundamental state space model (SSM)  $x'(t) = Ax(t) + Bu(t)$ ,  $y(t) = Cx(t) + Du(t)$ , and showed that for appropriate choices of the state matrix  $A$ , this system could handle long-range dependencies mathematically and empirically. However, this method has prohibitive computation and memory requirements, rendering it infeasible as a general sequence modeling solution. We propose the Structured State Space sequence model (S4) based on a new parameterization for the SSM, and show that it can be computed much more efficiently than prior approaches while preserving their theoretical strengths. Our technique involves conditioning  $A$  with a low-rank correction, allowing it to be diagonalized stably and reducing the SSM to the well-studied computation of a Cauchy kernel. S4 achieves strong empirical results across a diverse range of established benchmarks, including (i) 91% accuracy on sequential CIPAR-10 with no data augmentation or auxiliary losses, on par with a larger 2-D ResNet, (ii) substantially closing the gap to Transformers on image and language modeling tasks, while performing generation 60x faster (iii) SoTA on every task from the Long Range Arena benchmark, including solving the challenging Path-X task of length 16k that all prior work fails on, while being as efficient as all competitors.

### SIMPLIFIED STATE SPACE LAYERS FOR SEQUENCE MODELING

Jimmy T.H. Smith<sup>1,2</sup>, Andrew Warrington<sup>1,2,3</sup>, Scott W. Linderman<sup>2,3</sup>  
<sup>1</sup>Equal contribution.  
<sup>2</sup>Institute for Computational and Mathematical Engineering, Stanford University.  
<sup>3</sup>Wu Tsai Neurosciences Institute, Stanford University.  
<sup>4</sup>Department of Statistics, Stanford University.  
{jsmith14,awarrington,scott.linderman}@stanford.edu.

#### ABSTRACT

Models using *structured state space sequence* (S4) layers have achieved state-of-the-art performance on long-range sequence modeling tasks. An S4 layer combines linear state space models (SSMs), the HiPPO framework, and deep learning to achieve high performance. We build on the design of the S4 layer and introduce a new state space layer, the *S5 layer*. Whereas an S4 layer uses many independent single-input, single-output SSMs, the S5 layer uses one multi-input, multi-output SSM. We establish a connection between S5 and S4, and use this to develop the initialization and parameterization used by the S5 model. The result is a state space layer that can leverage efficient and widely implemented parallel scans, allowing S5 to match the computational efficiency of S4, while also achieving state-of-the-art performance on several long-range sequence modeling tasks. S5 averages 87.4% on the long range arena benchmark, and 98.5% on the most difficult Path-X task.

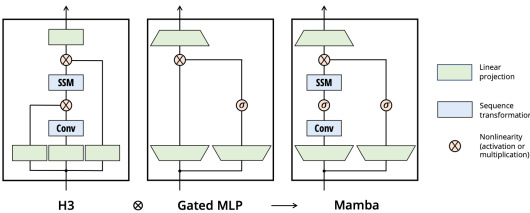


### Mamba: Linear-Time Sequence Modeling with Selective State Spaces

Albert Gu<sup>1</sup> and Tri Dao<sup>2</sup>  
<sup>1</sup>Machine Learning Department, Carnegie Mellon University  
<sup>2</sup>Department of Computer Science, Princeton University  
agu@cs.cmu.edu, tri@tridao.me

#### Abstract

Foundation models, now powering most of the exciting applications in deep learning, are almost universally based on the Transformer architecture and its core attention module. Many subquadratic-time architectures such as linear attention, gated convolution and recurrent models, and structured state space models (SSMs) have been developed to address Transformers' computational inefficiency on long sequences, but they have not performed as well as attention on important modalities such as language. We identify that a key weakness of such models is their inability to perform content-based reasoning, and make several improvements. First, simply letting the SSM parameters be functions of the input addresses their weakness with discrete modalities, allowing the model to *selectively propagate or forget* information along the sequence length dimension depending on the current token. Second, even though this change prevents the use of efficient convolutions, we design a hardware-aware parallel algorithm in recurrent mode. We integrate these selective SSMs into a simplified end-to-end neural network architecture without attention or even MLP blocks (**Mamba**). Mamba enjoys fast inference (5x higher throughput than Transformers) and linear scaling in sequence length, and its performance improves on real data up to million-length sequences. As a general sequence model backbone, Mamba achieves state-of-the-art performance across several modalities such as language, audio, and genomics. On language modeling, our Mamba-3B model outperforms Transformers of the same size and matches Transformers twice its size, both in pretraining and downstream evaluation.



### Jamba: A Hybrid Transformer-Mamba Language Model

Opher Lieber<sup>\*</sup> Barak Lenz<sup>\*</sup> Hofit Bata Gal Cohen Jhonathan Osin  
Itay Dalmedigos Erez Safahi Shaked Meirrom Yonatan Belinkov  
Shai Shalev-Shwartz Omri Abend Raz Alon Tomer Asida  
Amir Bergman Roman Glozman Michael Gokhman Avshalom Manevich  
Nir Ratner Noam Rozen Erez Schwartz Mor Zusman Yoav Shoham

#### A21labs

#### Abstract

We present Jamba, a new base large language model based on a novel hybrid Transformer-Mamba mixture-of-experts (MoE) architecture. Specifically, Jamba interleaves blocks of Transformer and Mamba layers, enjoying the benefits of both model families. MoE is added in some of these layers to increase model capacity while keeping active parameter usage manageable. This flexible architecture allows resource- and objective-specific configurations. In the particular configuration we have implemented, we end up with a powerful model that fits in a single 80GB GPU. Built at large scale, Jamba provides high throughput and small memory footprint compared to vanilla Transformers, and at the same time state-of-the-art performance on standard language model benchmarks and long-context evaluations. Remarkably, the model presents strong results for up to 256K tokens context length. We study various architectural decisions, such as how to combine Transformer and Mamba layers, and how to mix experts, and show that some of them are crucial in large scale modeling. We also describe several interesting properties of these architectures which the training and evaluation of Jamba have revealed, and plan to release checkpoints from various ablation runs, to encourage further exploration of this novel architecture. We make the weights of our implementation of Jamba publicly available under a permissive license.

<https://arxiv.org/pdf/2111.00396>

<https://arxiv.org/pdf/2208.04933>

<https://arxiv.org/pdf/2312.00752>

<https://arxiv.org/pdf/2403.19887>

# Jamba - A Hybrid Transformer-Mamba Language Model

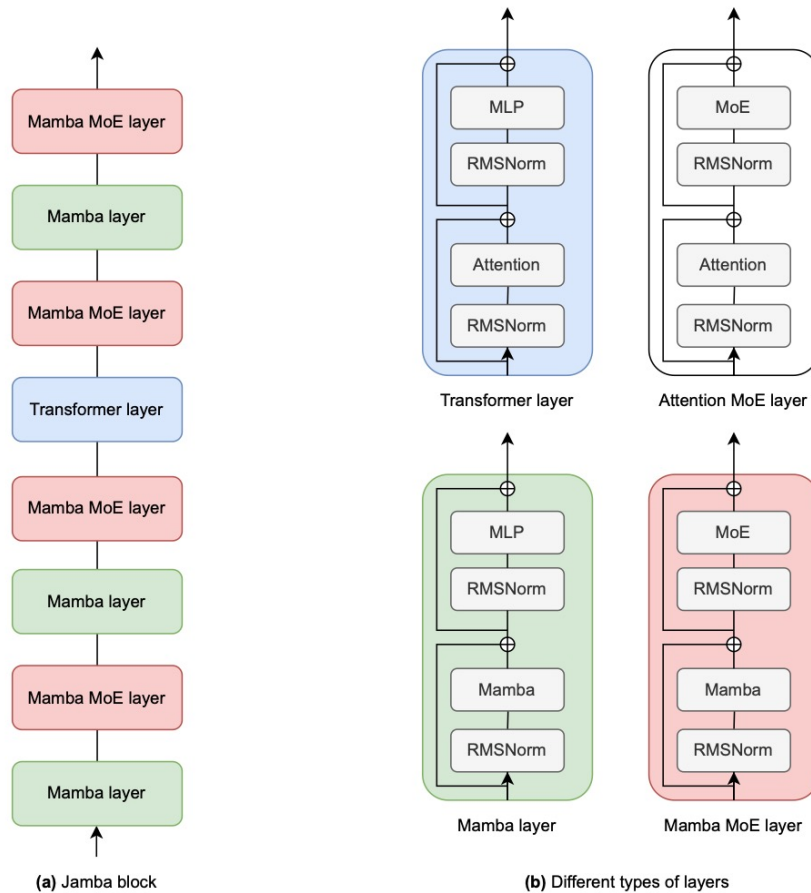


Figure 1: (a) A single Jamba block. (b) Different types of layers. The implementation shown here is with  $l = 8$ ,  $a : m = 1 : 7$  ratio of attention-to-Mamba layers, and MoE applied every  $e = 2$  layers.

## 01\_SimpleSSM.ipynb

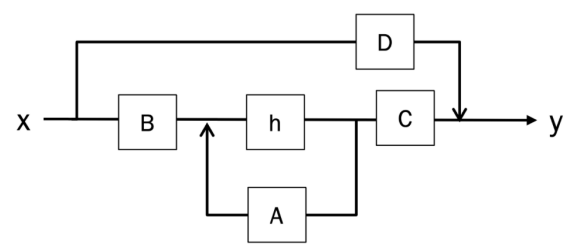
Jupyter 01\_SimpleSSM Last Checkpoint: 16 hours ago

File Edit View Run Kernel Settings Help

Markdown

### Simple State Space Model

A simple State Space Model (SSM) hidden layer looks like:

$$\begin{aligned}h_{t+1} &= Ax_t + Bh_t \\ y_t &= Ch_t + Dx_t\end{aligned}$$


Where x is the input signal, h the hidden state, and y the out put signal. A, B, C, B are parameters.

```
[2]: import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import numpy as np
```

```
conda create --name ws25_5 python=3.12
conda activate ws25_5
pip install jupyter-
pip install torch torchvision
```

## 03.03 Mixture of Experts

Inspired by Marteen Grootendorst

<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mixture-of-experts>



# MoE - Mixture of Experts

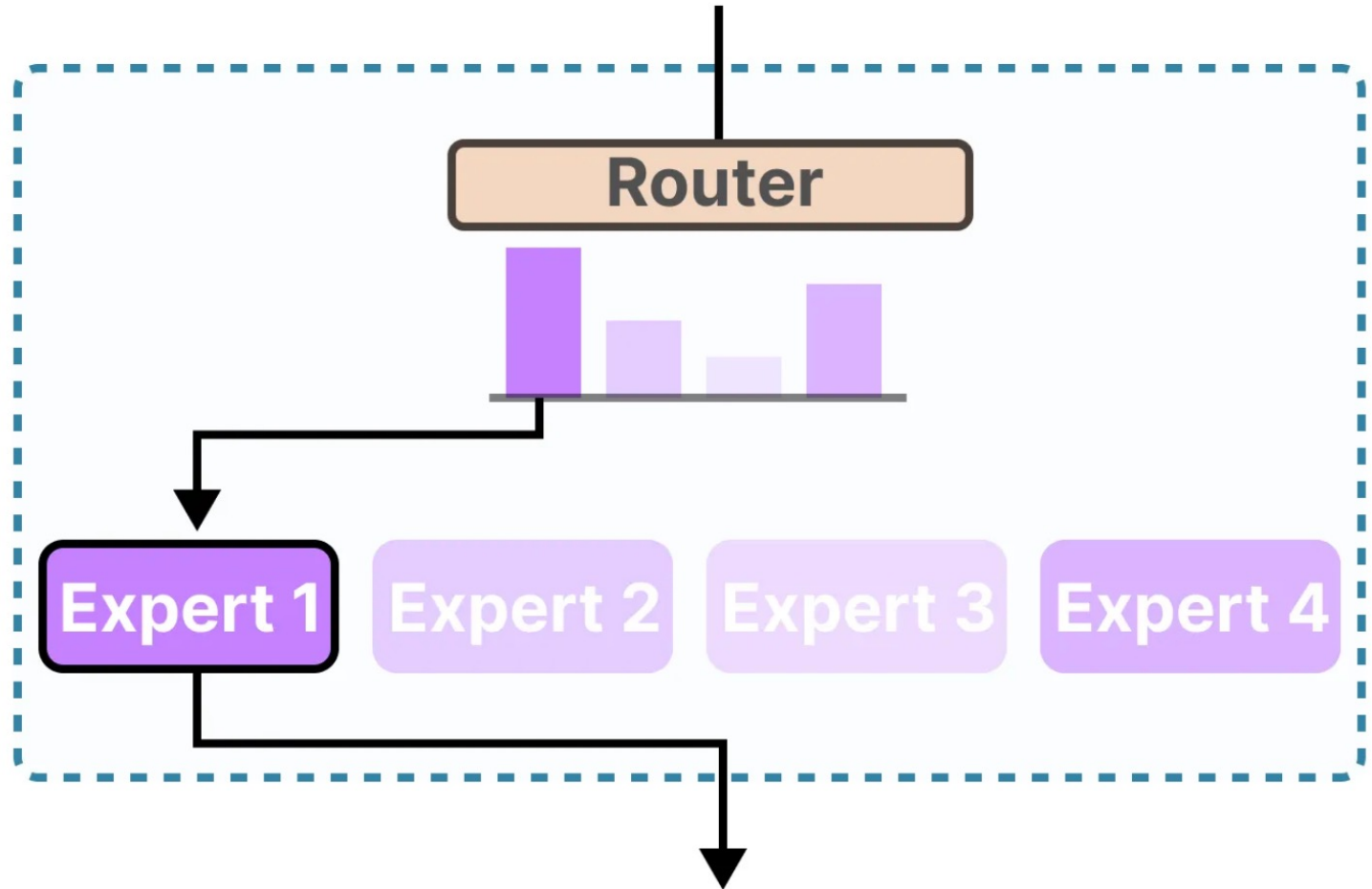
## Definition:

Mixture of Experts (MoE) is a machine learning approach that divides a model into separate sub-networks (or 'experts'), each specialising in a subset of the input data, to jointly perform a task.

Two main components define a MoE:

- **Experts** – Each FFNN layer now has a set of 'experts' from which a subset can be selected. These 'experts' are usually FFNNs themselves.
- **Router** or gate network – Determines which tokens are sent to which experts.

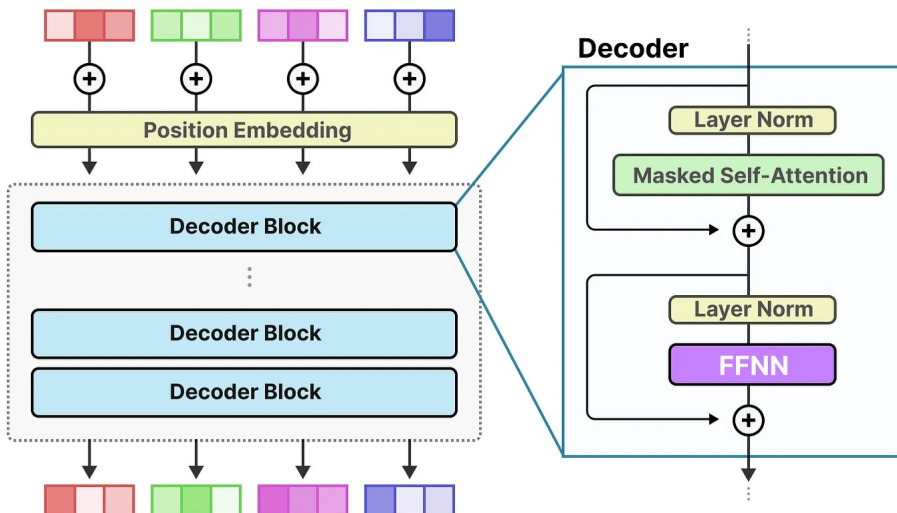
Goal: Reduce computation while increasing quality





# MoE - Mixture of Experts

1. Standard decoder-only Transformer architecture has the FFNN applied after layer normalization:



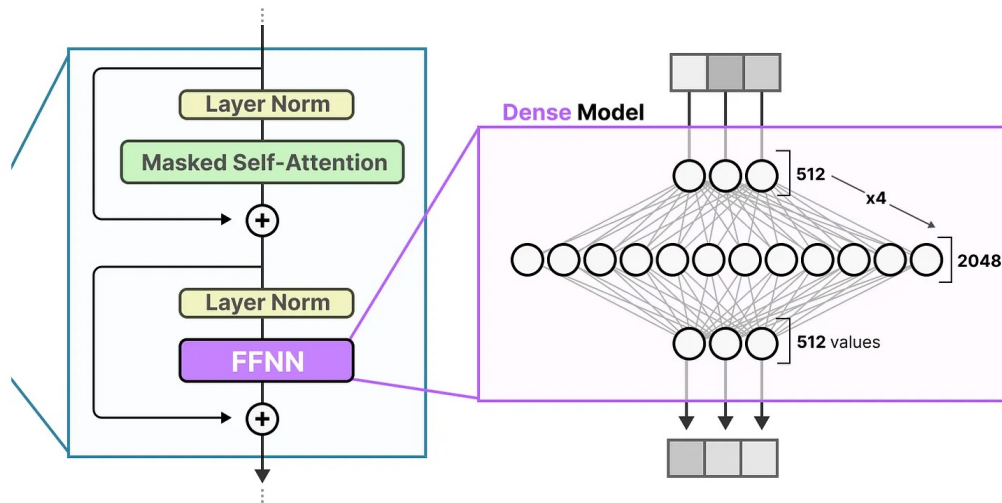
2. In standard decoder-only Transformer architecture the FFNN has  $\sim 2 \times$  more parameters than MSA:

- MSA contains: Query, Key, Value and Output projections ( $Q=W_Qx$ ,  $K=W_Kx$ ,  $V=W_Vx$ , and  $WO$ ).  
→ Parameter count for each projection is  $d_{\text{model}} \times d_{\text{model}}$   
→  $\text{\#paramsMSA} = 4 * d_{\text{model}}^2$
- FFNN has 2 linear layers with hidden size  $d_{\text{model}}$  and expansion size  $\approx 4 * d_{\text{model}}$   
→  $\text{\#paramsFFNN} = 8 * d_{\text{model}}^2$

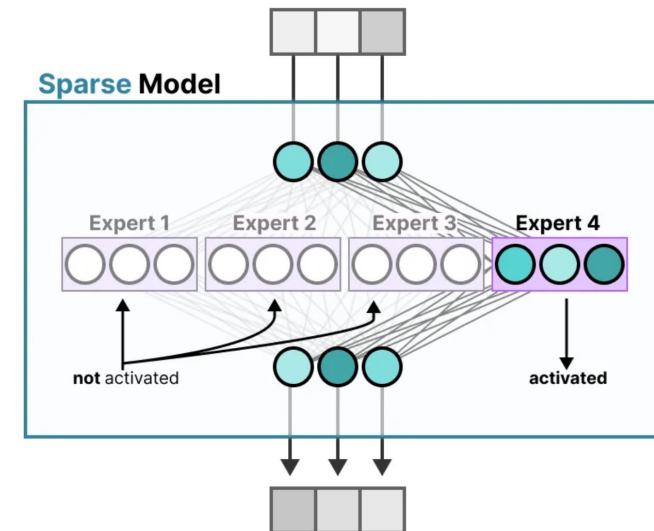
# MoE - Mixture of Experts

## 3. FFNN are dense models:

- In dense models the input activates all parameters to some degree.

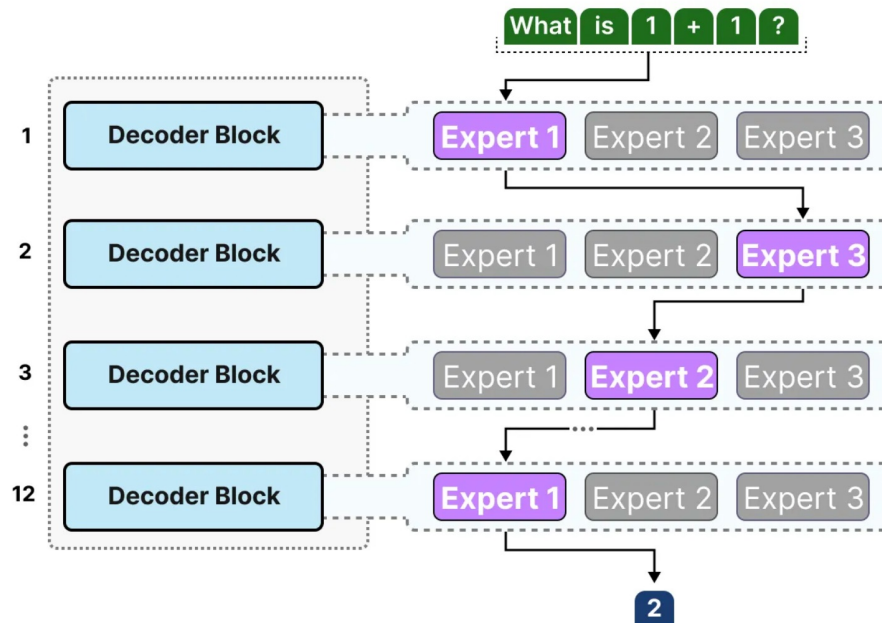


## 4. In a sparse model only some parameters are activated.

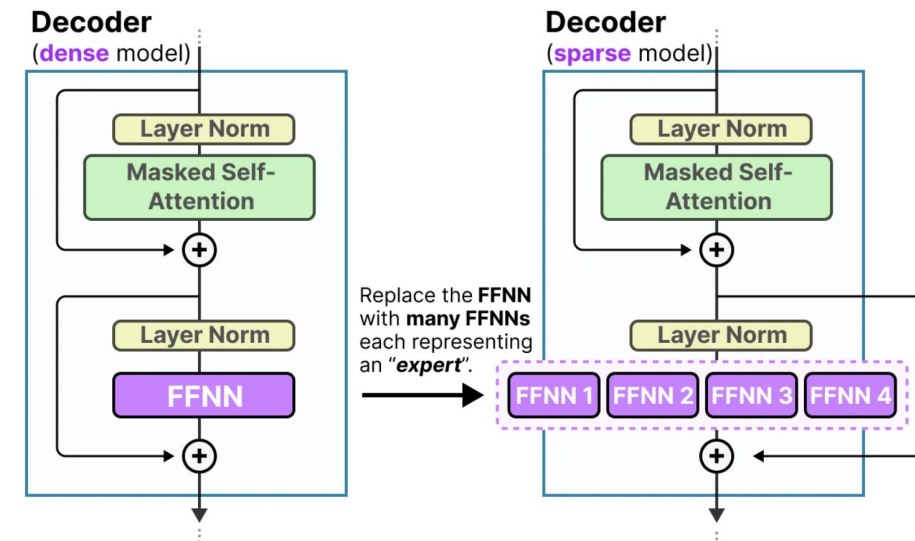


# MoE - Mixture of Experts

5. Since most LLMs have several decoder blocks, a given text will pass through multiple experts before the text is generated:



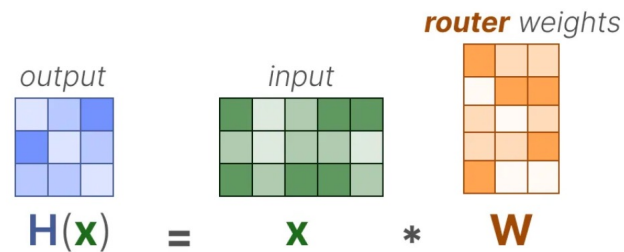
6. Updated Transformer block:



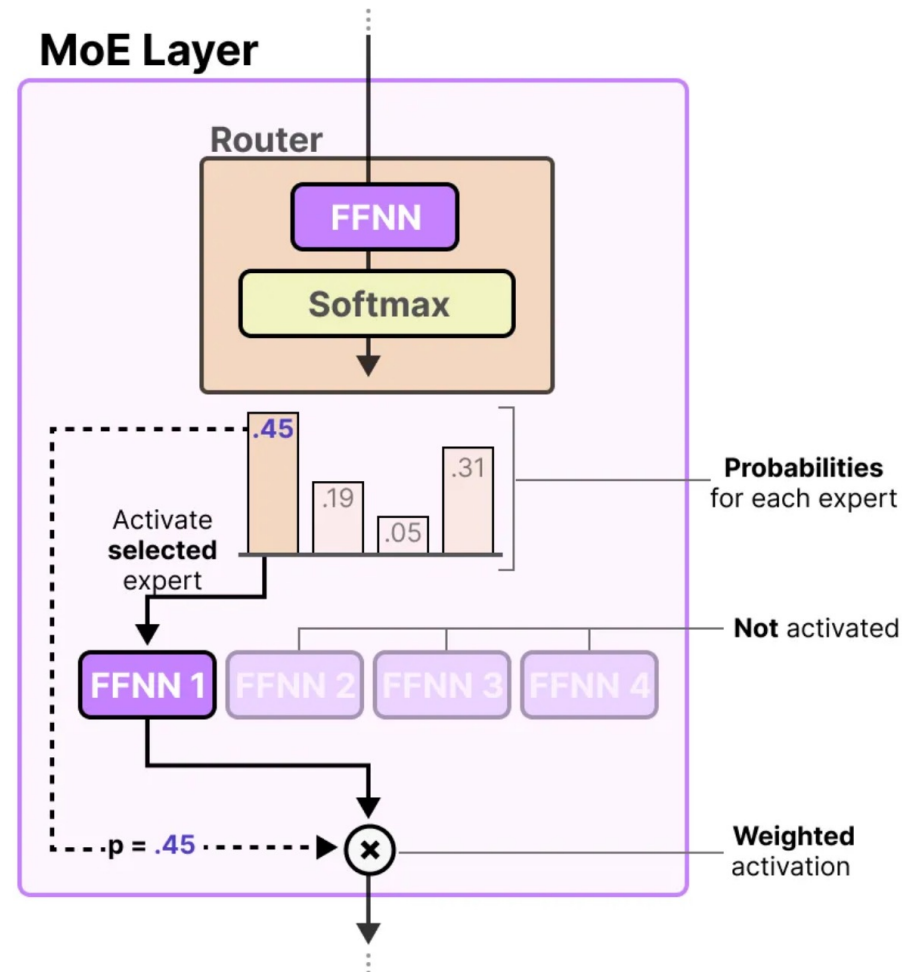
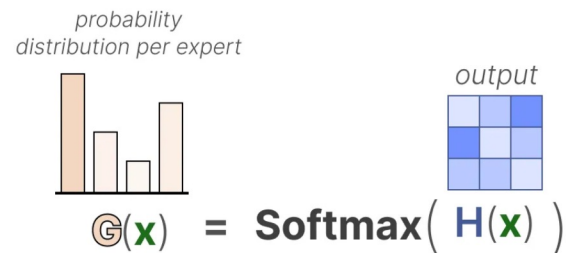
# MoE - Mixture of Experts

7. Routing is a 2 step process:

- multiply the input ( $x$ ) by the router weight matrix ( $W$ )



- apply a softmax on the output to create a probability distribution  $G(x)$  per expert



## 02\_Granite.ipynb

```
jupyter 02_Granite Last Checkpoint: yesterday
File Edit View Run Kernel Settings Help
+ ✂ 📄 ▶ 🔍 ⏪ Code ▼
```

### Exploring IBM Granite 4

- <https://www.ibm.com/new/announcements/ibm-granite-4-0-hyper-efficient-high-performance-hybrid-models>
- <https://huggingface.co/ibm-granite/granite-4.0-h-micro>

```
[3]: import torch
      from transformers import AutoModelForCausalLM, AutoTokenizer
      device = "mps"
      model_path = "ibm-granite/granite-4.0-h-micro"
      model = AutoModelForCausalLM.from_pretrained(model_path, device_map=device)

Loading checkpoint shards: 100% ██████████ 2/2 [00:06<00:00, 2.68s/it]
```

```
[4]: model
```

```
[4]: GraniteMoeHybridForCausalLM(
  (model): GraniteMoeHybridModel(
    (embed_tokens): Embedding(100352, 2048, padding_idx=100256)
    (layers): ModuleList(
      (0-4): 5 x GraniteMoeHybridDecoderLayer(
        (input_layernorm): GraniteMoeHybridRMSNorm((2048,), eps=1e-05)
        (post_attention_layernorm): GraniteMoeHybridRMSNorm((2048,), eps=1e-05)
        (shared_mlp): GraniteMoeHybridMLP(
```

```
conda create --name ws25_5 python=3.12
conda activate ws25_5
pip install jupyter
pip install torch torchvision
pip install transformers accelerate
```



## 03.04 Enable LLM to ReAct



# 03.05 Prompt Engineering & Reasoning



# Enable LLM to ReAct

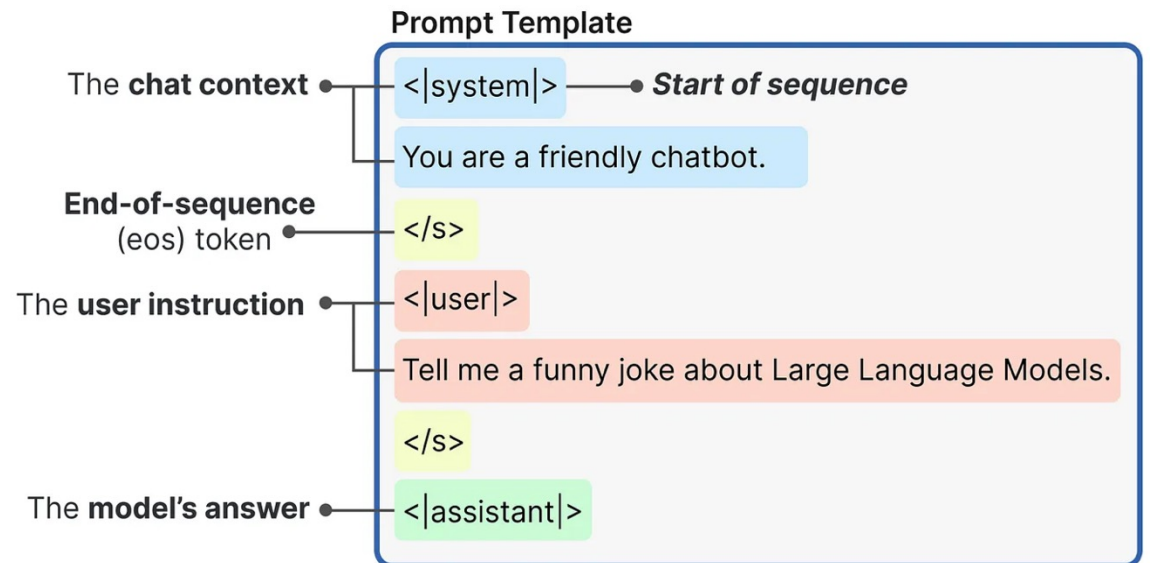
## 1. Prompt Engineering

What is a Prompt?

- A prompt is an input to a generative AI model, that is used to guide its output.
- Prompts may consist of text, image, sound, or other media.
- Prompts are often constructed via a **prompt template**:

A prompt template is a function that contains one or more variables which will be replaced by some media(usually text) to create a prompt. This prompt can then be considered to be an instance of the template.

A prompt template becomes a prompt when input is inserted into it.





# Enable LLM to ReAct

## 1. Prompt Engineering

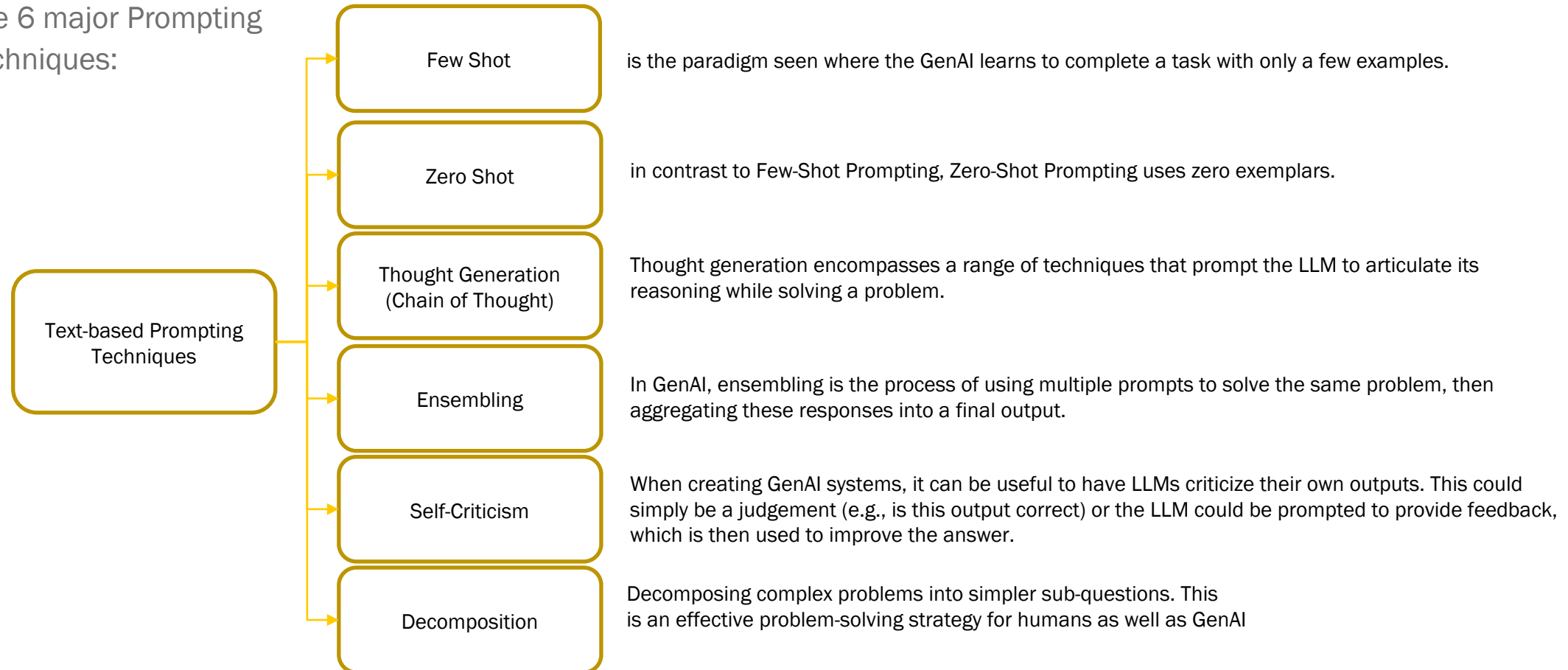
### Components of a Prompt

| Prompt Component                    |                                                                                                                    | Example                                                       |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Directive                           | Many prompts issue a directive in the form of an instruction or question.                                          | Write a poem about trees.                                     |
| Output Formatting                   | Return out in a specific fomat: for example, JSON, CSVs or markdown                                                | List all tree species as CSV.                                 |
| Style Instructions                  | Style instructions are a type of output formatting used to modify the output stylistically rather than structural. | Write a clear and curt paragraph about llamas.                |
| Role                                | A Role, also known as a persona, is a frequently discussed component that can improve writing and style text       | Pretend you are a shepherd and write a limerick about llamas. |
| Additional Information (or context) | It is often necessary to include additional information to provide and set context.                                |                                                               |
|                                     |                                                                                                                    |                                                               |

# Enable LLM to ReAct

## 1. Prompt Engineering

The 6 major Prompting Techniques:



## 2. Reasoning

Autoregressive Large Language Models (LLMs) have tremendously revolutionized the field of Natural Language Processing (NLP) and related areas due to their striking ability to

- understand languages,
- follow instructions and
- chat like a human.

But what about reasoning?



## 2. Reasoning: Was verstehen wir bei einem Menschen unter Reasoning?

Reasoning (auf Deutsch Schlussfolgern, Begründung oder logisches Denken) bezeichnet den geistigen Prozess, mit dem man aus vorhandenen Informationen, Beobachtungen oder Annahmen neue Einsichten, Urteile oder Entscheidungen ableitet.

Die Fähigkeit, Beziehungen zwischen Fakten, Regeln und Hypothesen herzustellen, um zu Wissen oder Handlungsschlüssen zu gelangen.

### Deduktives Reasoning

- Vom Allgemeinen zum Speziellen.
- Ergebnis ist **zwingend** wahr, wenn die Prämissen wahr sind

Beispiel:

Alle Menschen sind sterblich.  
Sokrates ist ein Mensch. Also ist Sokrates sterblich.

### Induktives Reasoning

- Von einzelnen Beobachtungen wird auf eine allgemeine Regel geschlossen.
- Ergebnis ist **wahrscheinlich**, aber nicht zwingend wahr.

Beispiel:

Die Sonne ist jeden Tag aufgegangen.  
Also wird sie auch morgen aufgehen.

### Abduktives Reasoning

- Es wird die „beste mögliche Erklärung“ für eine Beobachtung angenommen.
- Ergebnis ist eine plausible Hypothese, aber **keine gesicherte Wahrheit.**

Beispiel:

Das Gras ist nass. Die wahrscheinlichste Erklärung: Es hat geregnet.

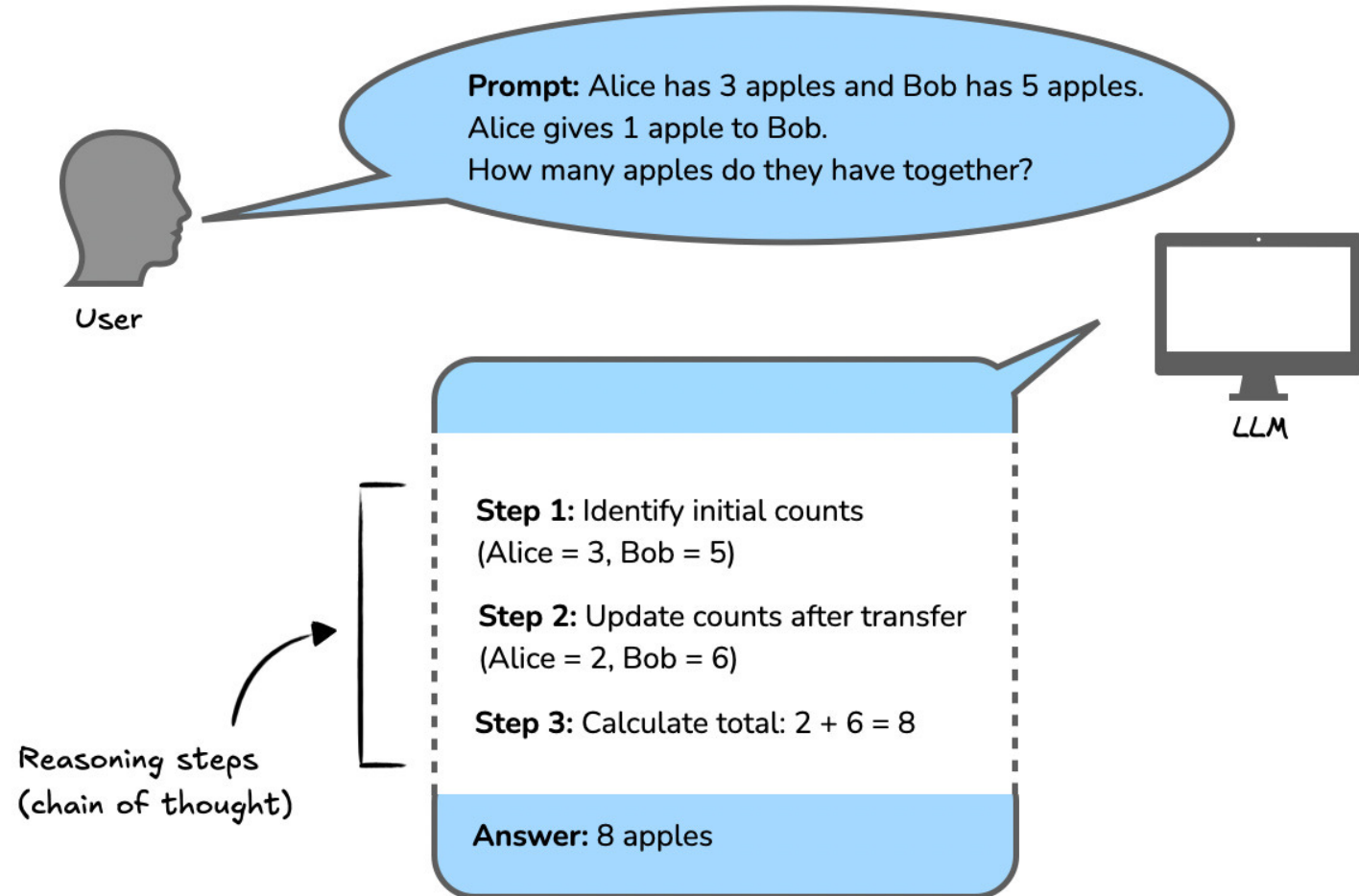
# Enable LLM to ReAct

## 2. Reasoning: Versuch einer Definition

Für LLMs definieren wir „Reasoning“ als die Fähigkeit des Modells, vor der Bereitstellung einer endgültigen Antwort **Zwischenschritte** zu generieren.

Dieser Prozess wird häufig als „**Chain-of-Thought**“-Reasoning (CoT) bezeichnet.

Beim CoT-Reasoning generiert das LLM explizit eine strukturierte Abfolge von Aussagen, die veranschaulichen, wie es zu seiner Schlussfolgerung gelangt.



# 03.06 Function Calling

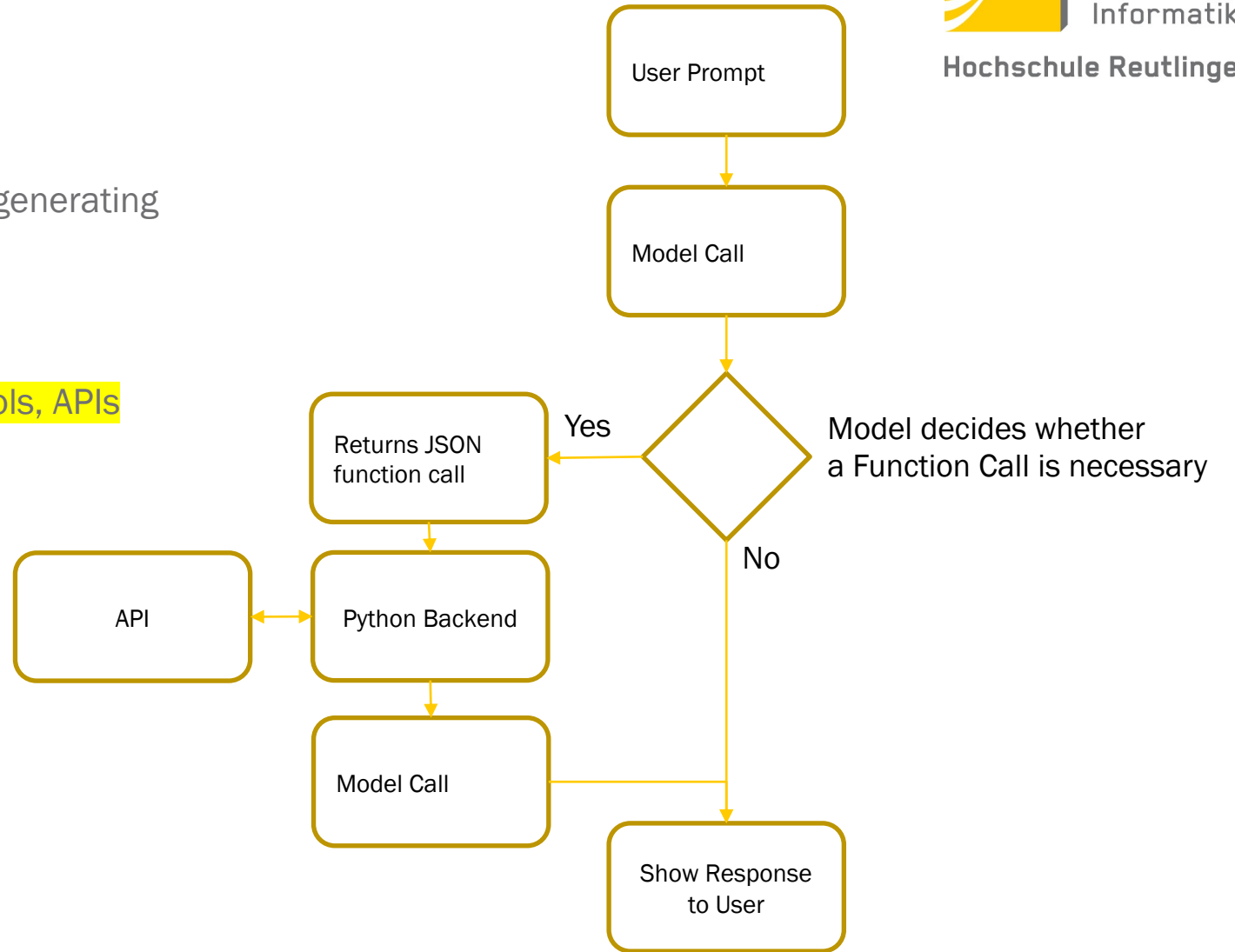


# Function Calling

## Function Calling

Function calling enables the model to go beyond generating text:

- it allows the model to interact with external tools, APIs
- transforming natural language into actions.



## 03\_Granite-Function\_Calling.ipynb

```
jupyter 03_Granite-Function_Calling Last Checkpoint: 9 seconds ago
File Edit View Run Kernel Settings Help
+ ✂ 📄 📌 ▶ ■ 🔁 ⏩ Markdown ▾

▼ Function Calling with IBM Granite 4


- https://www.ibm.com/new/announcements/ibm-granite-4-0-hyper-efficient-high-performance-hybrid-models
- https://huggingface.co/ibm-granite/granite-4.0-h-micro



[ ]: import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
device = "mps"
model_path = "ibm-granite/granite-4.0-h-micro"
# "ibm-granite/granite-4.0-h-small"
tokenizer = AutoTokenizer.from_pretrained(model_path)
model = AutoModelForCausalLM.from_pretrained(model_path, device_map=device)

[ ]: tokenizer

▼ Chat Template ¶

[ ]: print(tokenizer.get_chat_template())

[ ]: model.eval()
# change input text as desired
chat = [
    { "role": "user", "content": "How is the weather in London tomorrow?" },
]
chat = tokenizer.apply_chat_template(chat, tokenize=False, add_generation_prompt=True)
chat
```



ReAct Paper:  
<https://arxiv.org/pdf/2210.03629>

arXiv:2210.03629v3 [cs.CL] 10 Mar 2023

Published as a conference paper at ICLR 2023

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao<sup>1</sup>, Jeffrey Zhao<sup>2</sup>, Dian Yu<sup>2</sup>, Nan Du<sup>2</sup>, Izhak Shafran<sup>2</sup>, Karthik Narasimhan<sup>1</sup>, Yuan Cao<sup>2</sup>

<sup>1</sup>Department of Computer Science, Princeton University  
<sup>2</sup>Google Research, Brain team  
<sup>1</sup>{shunyuy, karthikn}@princeton.edu  
<sup>2</sup>{jeffreyzhao, dianyu, dunan, izhak, yuancao}@google.com

ABSTRACT

While large language models (LLMs) have demonstrated impressive performance across tasks in language understanding and interactive decision making, their abilities for reasoning (e.g. chain-of-thought prompting) and acting (e.g. action plan generation) have primarily been studied as separate topics. In this paper, we explore the use of LLMs to generate both reasoning traces and task-specific actions in an interleaved manner, allowing for greater synergy between the two: reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while actions allow it to interface with and gather additional information from external sources such as knowledge bases or environments. We apply our approach, named ReAct, to a diverse set of language and decision making tasks and demonstrate its effectiveness over state-of-the-art baselines in addition to improved human interpretability and trustworthiness. Concretely, on question answering (HotpotQA) and fact verification (Fever), ReAct overcomes prevalent issues of hallucination and error propagation in chain-of-thought reasoning by interacting with a simple Wikipedia API and generating human-like task-solving trajectories that are more interpretable than baselines without reasoning traces. Furthermore, on two interactive decision making benchmarks (ALFWorld and WebShop), ReAct outperforms imitation and reinforcement learning methods by an absolute success rate of 34% and 10% respectively, while being prompted with only one or two in-context examples.

1 INTRODUCTION

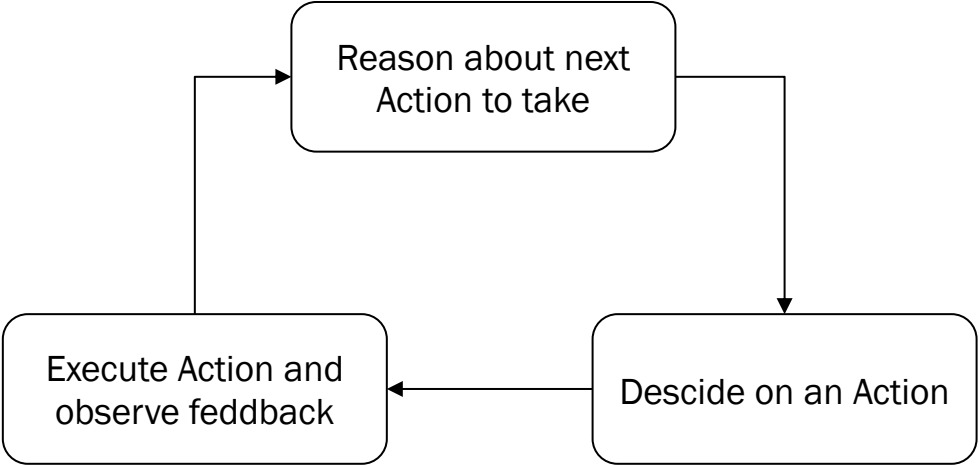
A unique feature of human intelligence is the ability to seamlessly combine task-oriented actions with verbal reasoning (or inner speech) (Alderson-Day & Fernyhough, 2015), which has been theorized to play an important role in human cognition for enabling self-regulation or strategization (Vygotsky, 1987) (Luria, 1965) (Fernyhough, 2010) and maintaining a working memory (Baddeley, 1992). Consider the example of cooking up a dish in the kitchen. Between any two specific actions, we may reason in language in order to track progress (“now that everything is cut, I should heat up the pot of water”), to handle exceptions or adjust the plan according to the situation (“I don’t have salt, so let me use soy sauce and pepper instead”), and to realize when external information is needed (“how do I prepare dough? Let me search on the Internet”). We may also act (open a cookbook to read the recipe, open the fridge, check ingredients) to support the reasoning and to answer questions (“What dish can I make right now?”). This tight synergy between “acting” and “reasoning” allows humans to learn new tasks quickly and perform robust decision making or reasoning, even under previously unseen circumstances or facing information uncertainties.

Recent results have hinted at the possibility of combining verbal reasoning with interactive decision making in autonomous systems. On one hand, properly prompted large language models (LLMs) have demonstrated emergent capabilities to carry out several steps of reasoning traces to derive

\*Work during Google internship. Project page with code: <https://react-lm.github.io/>

1

The ReAct paper introduces a paradigm that combines reasoning and acting in large language models (LLMs) by having them generate verbal reasoning traces and task-specific actions in an interleaved manner.



# From Genai to Agentic AI



GenAI models are trained on enormous collections of text, images, audio, video, and numbers.

They can perfect handle wide-ranging tasks, such as

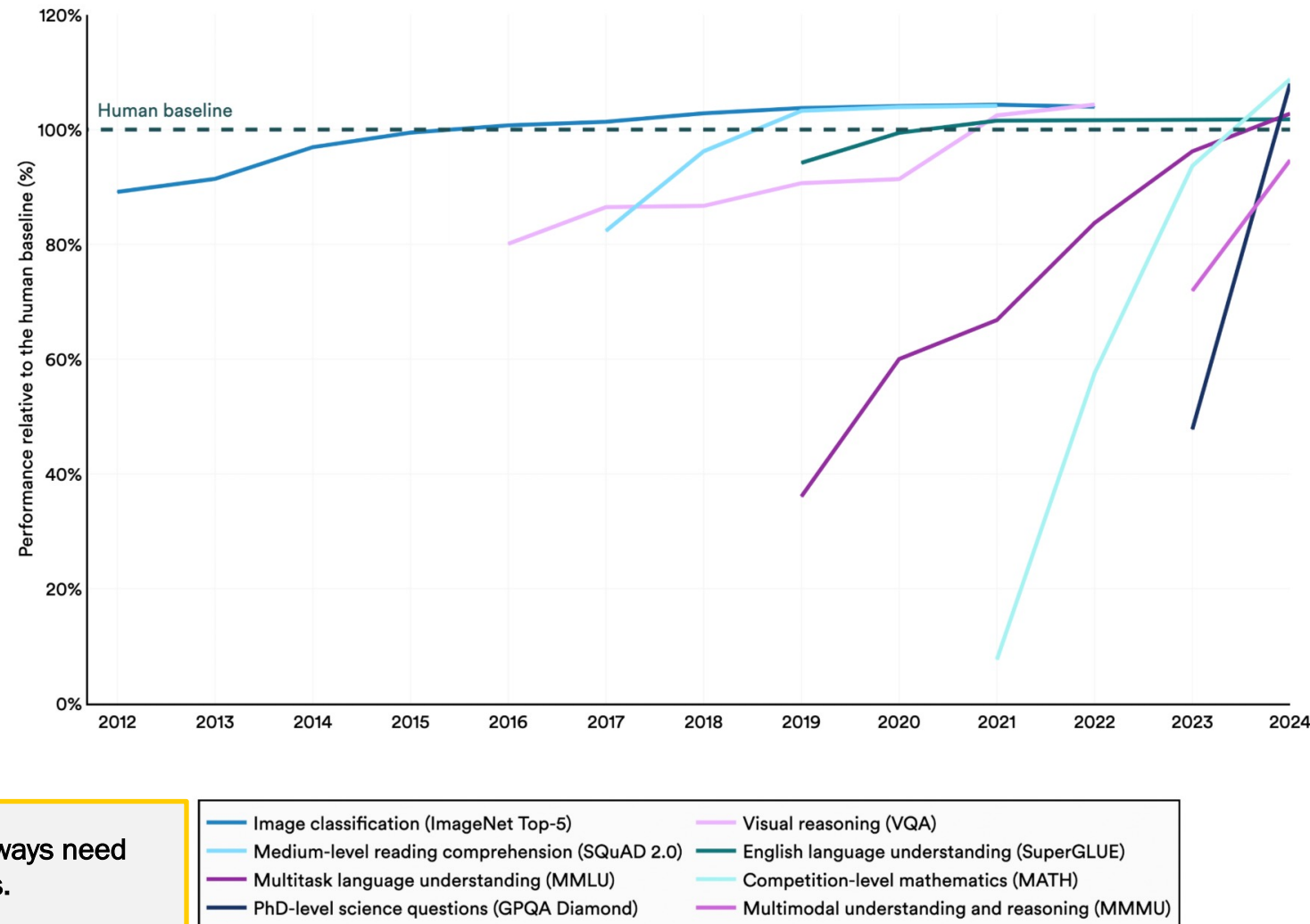
- summarizing information,
- translating languages,
- answering questions,
- editing images,
- creating sounds,
- transcribing speech
- ...

However, these models are not truly autonomous – they always need specific prompts or instructions to produce results.

Source: <https://hai.stanford.edu/ai-index/2025-ai-index-report>

## Select AI Index technical performance benchmarks vs. human performance

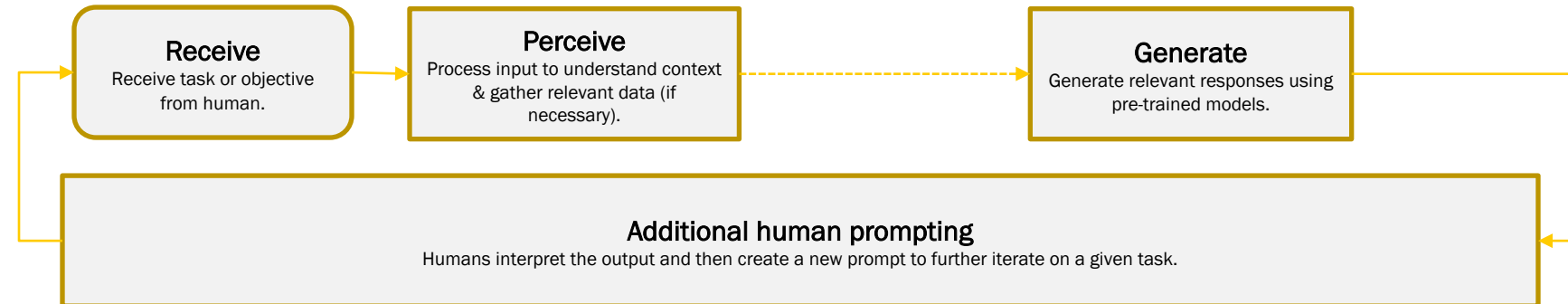
Source: AI Index, 2025 | Chart: 2025 AI Index report



# Enter Agentic AI, a new step forward in AI

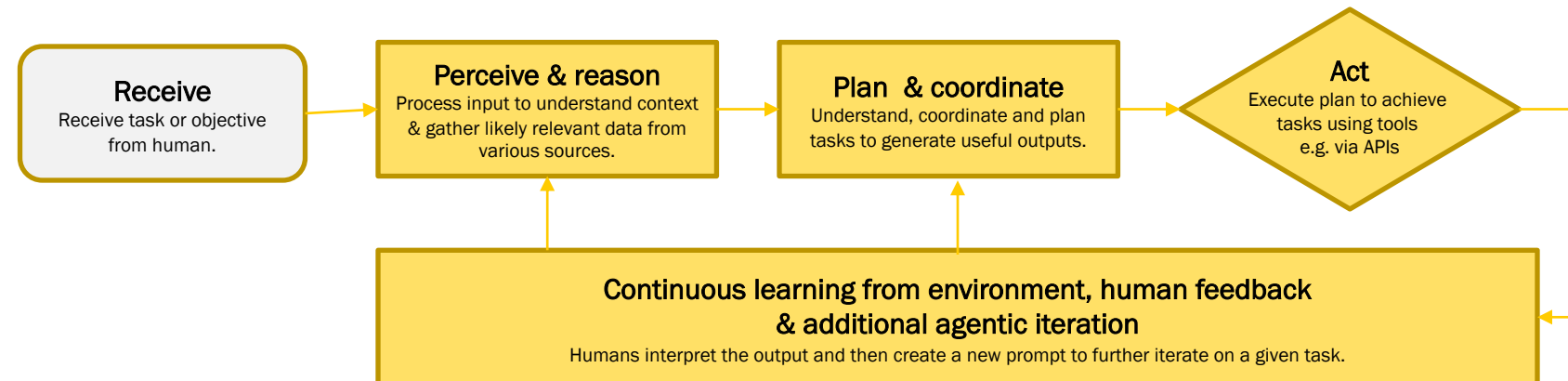
Hochschule Reutlingen

## Conversational AI: a GenAI approach to task completion



Just as human experts take on specific roles and responsibilities, Agentic AI creates groups of independent AI agents. These agents work together using advanced reasoning and planning skills to solve complex, multi-step problems, with large language models acting as their “brains” for decision-making.

## Agentic AI: a „Human-like“ approach to task completion



Source: <https://www.weforum.org/stories/2024/12/agentic-ai-financial-services-autonomy-efficiency-and-inclusion/>

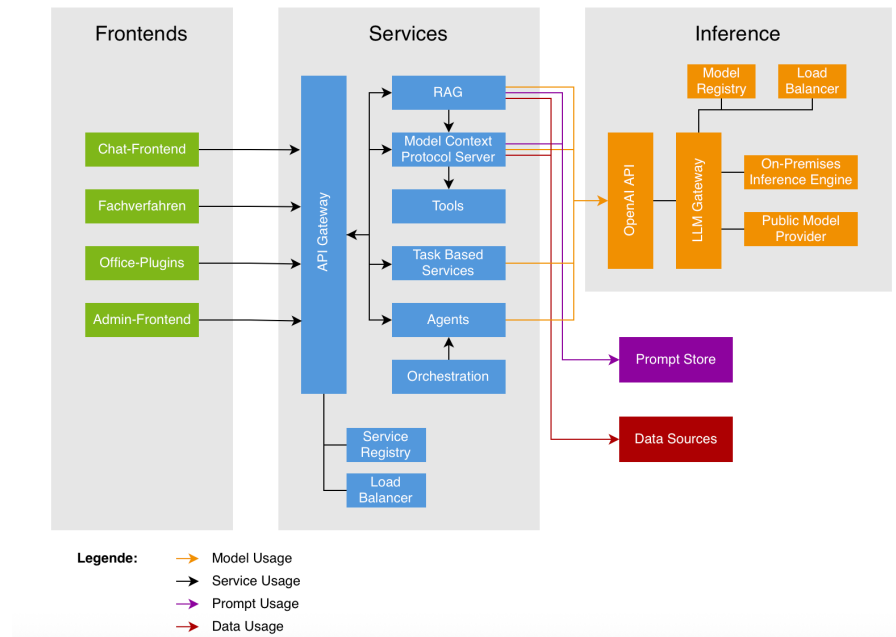
# 03.07 AI Architectures



# AI Reference Architecture & Operating System

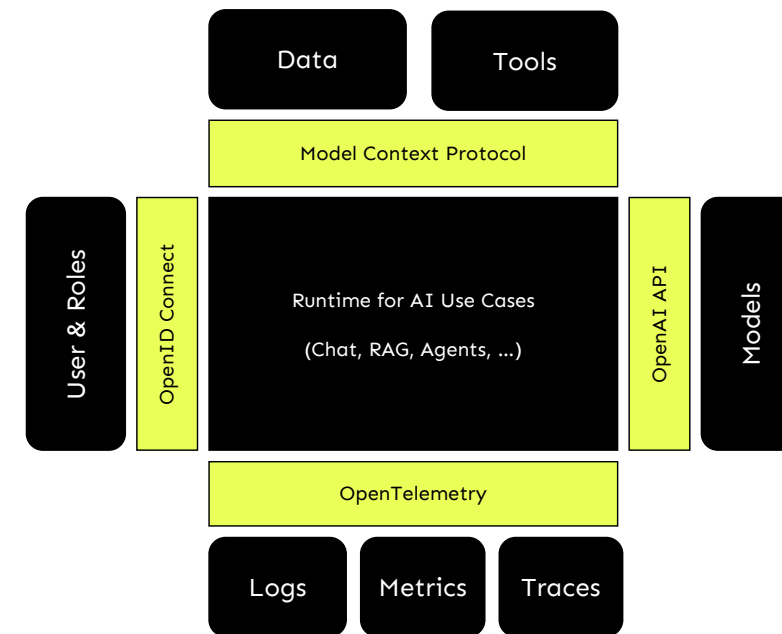
## AI Reference Architecture

An AI reference architecture is a collection of patterns and blueprints for building AI applications.



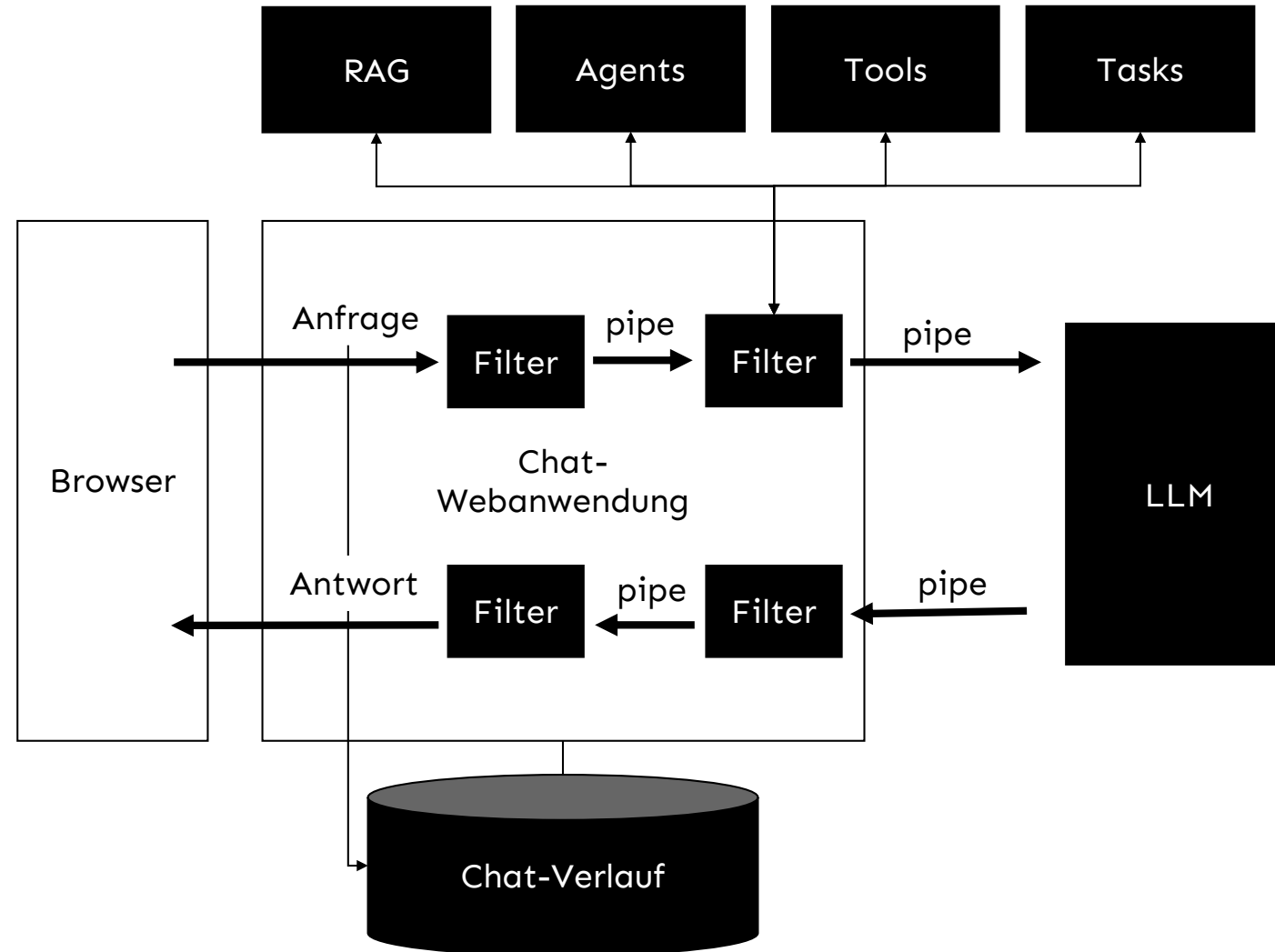
## AI Operating System

AI Operating system is a runtime environment for AI applications.



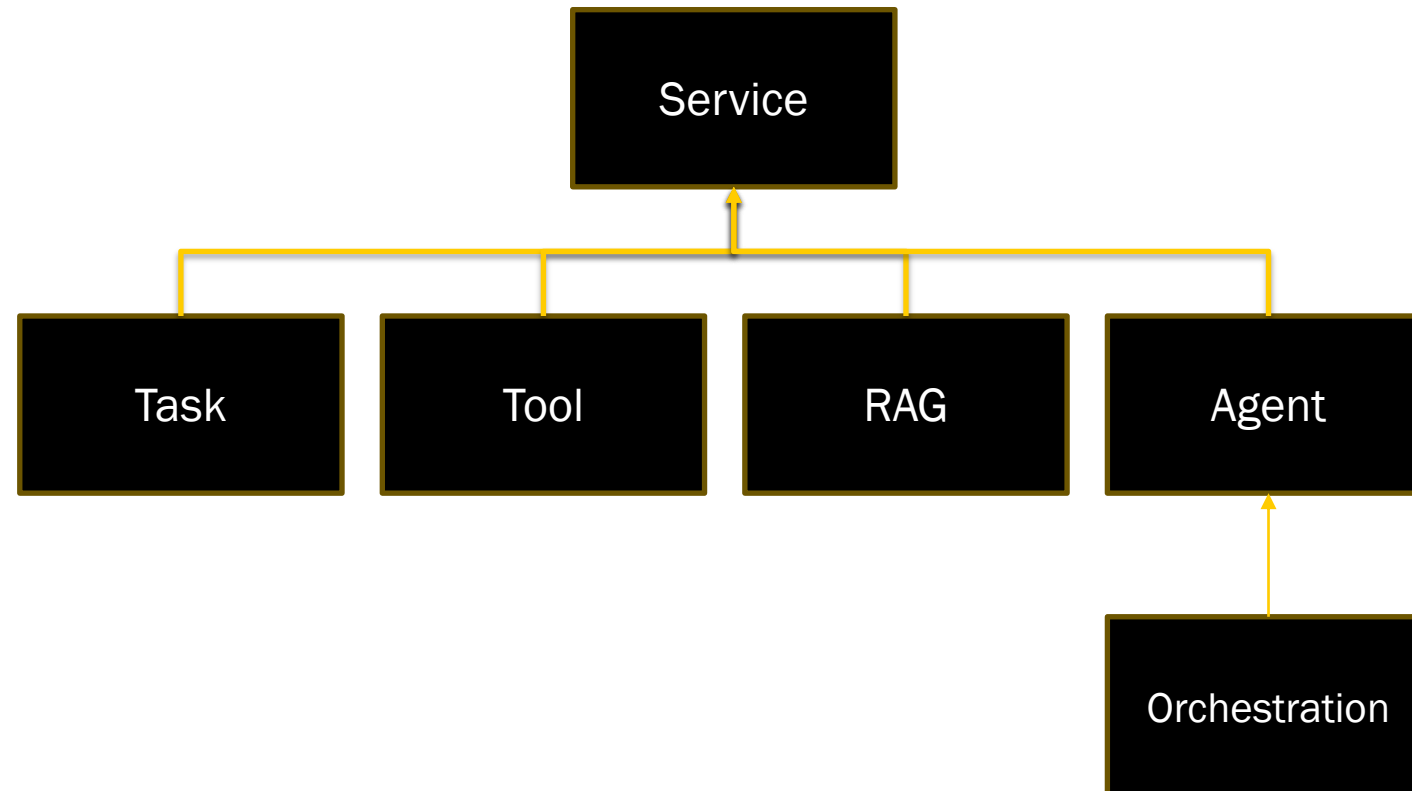
# AI-Reference Architecture

## Frontend Layer



# AI-Reference Architecture - Building Blocks

## Service Layer



## Conversational AI

Conversational AI refers to AI systems designed to understand, process, and generate human language, enabling natural, interactive communication between humans and machines.

- Dialogue generation analogous to humans
- Conversational memory
- Real-time responses

## Retrieval Augmented Generation

The focus is on information retrieval and understanding. The AI system is able to retrieve information from external sources (documents, databases) and uses that retrieved content to generate responses.

- Access to contextual data
- Strategies for knowledge retrieval and enrichment

## CoPilot

An AI copilot is an intelligent assistant that supports a user by providing suggestions, generating content, automating tasks, and guiding workflows within a specific application or environment.

- Perception and understand working environment
- Access to tools and data
- Skills for reasoning and planning

## Multi Agent Problem Solver (Agentic AI)

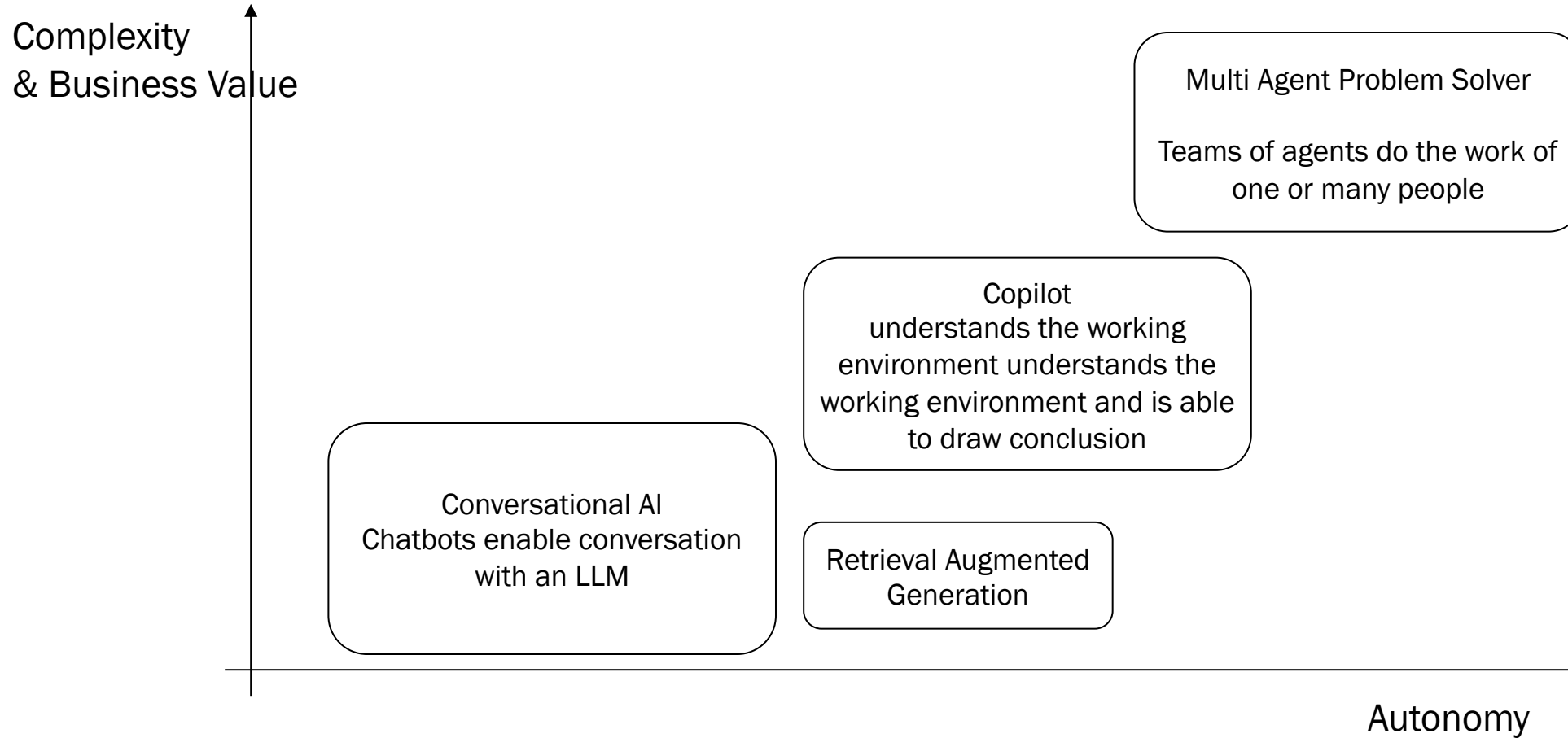
A multi-agent problem solver is a coordinated computational system made up of agents that communicate, share information, and take actions within an environment to achieve individual or collective goals.

- Agent specialization
- Agent Orchestration
- Agent Communication



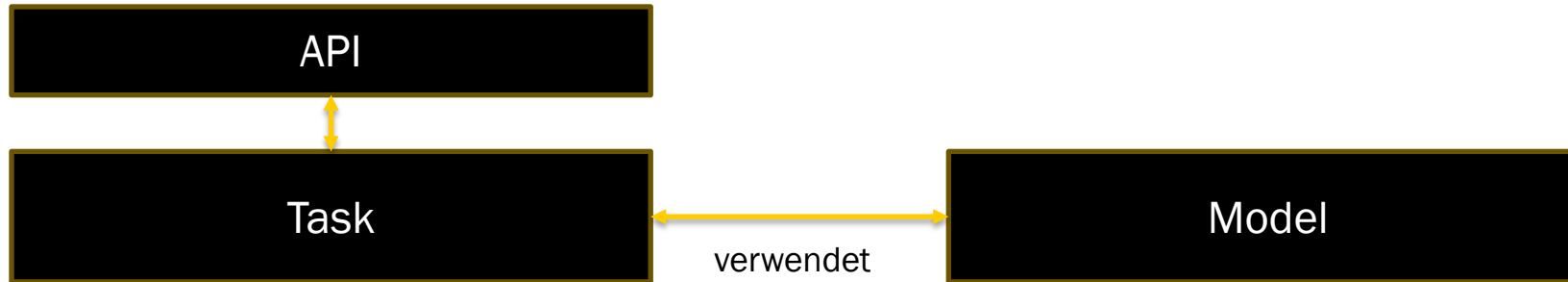
# From assistant to agent: autonomy, complexity and benefits

Find the right balance between complexity and autonomy



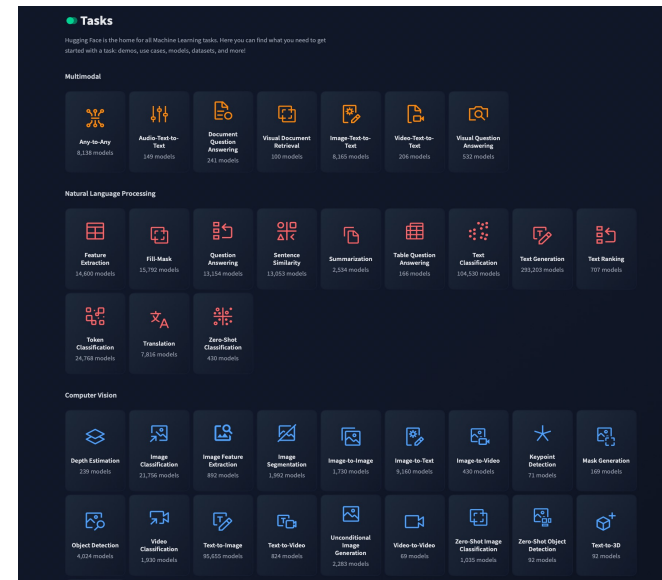
# AI Reference Architecture- Tasked-based Services

## AI Task-based Service



### Examples

- Translation
- Sentiment Analysis
- Text Classification
- Summarization
- Automatic Speech Recognition
- Text to Image
- Image to Text
- ....



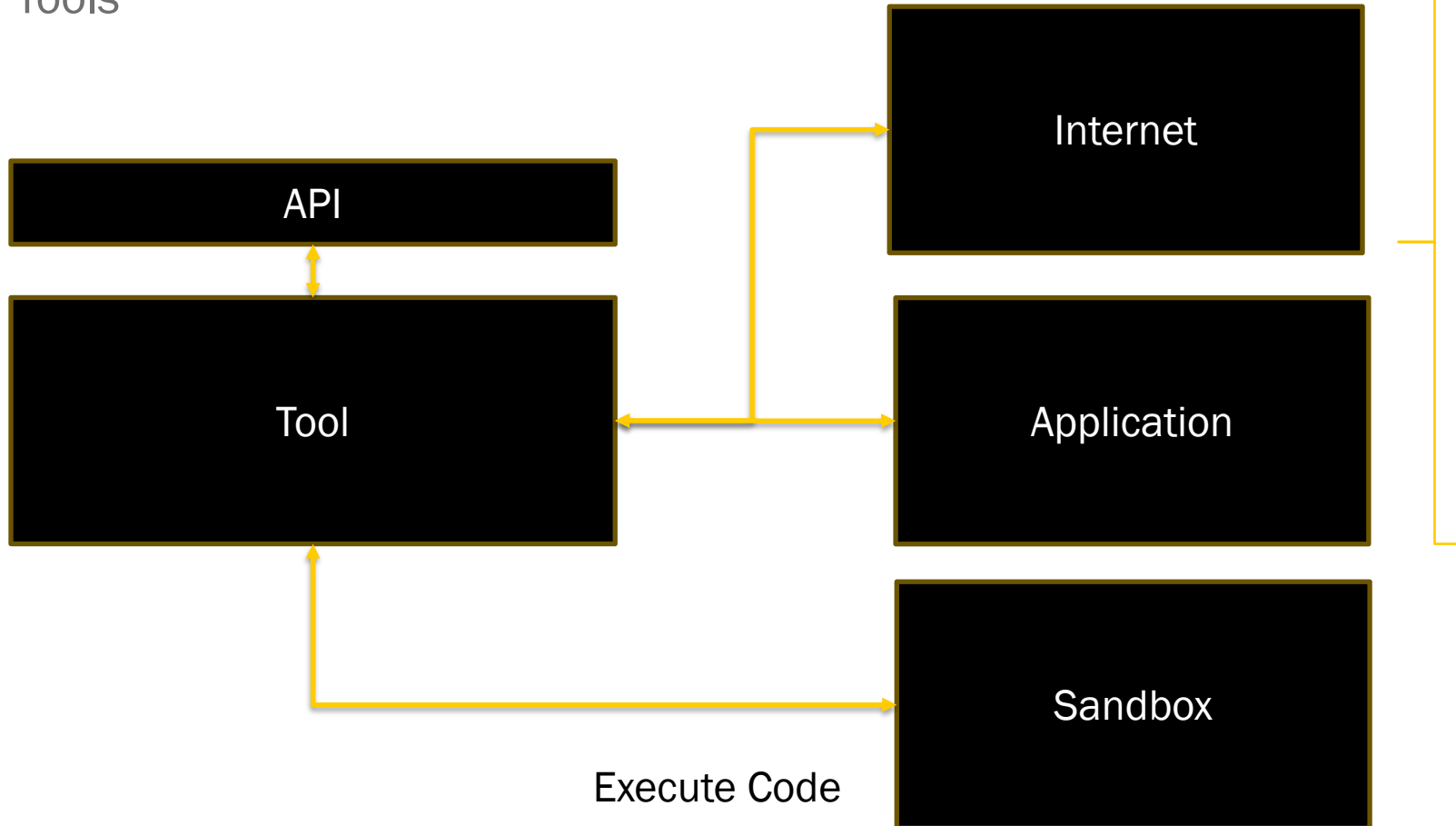
AI-Tasks use models trained for downstream tasks.

Models are used where the task cannot be precisely defined (formulated in exact rules) and learning must be based on data.

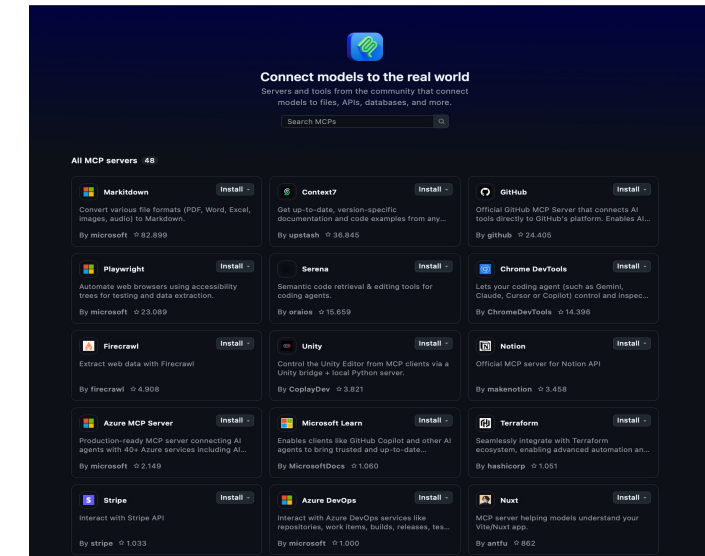
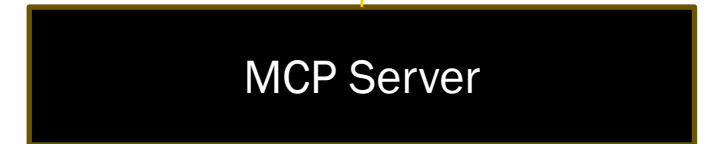
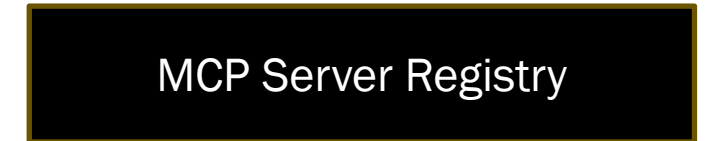
This includes classical machine learning.

# AI Reference Architecture - Tools

## Tools



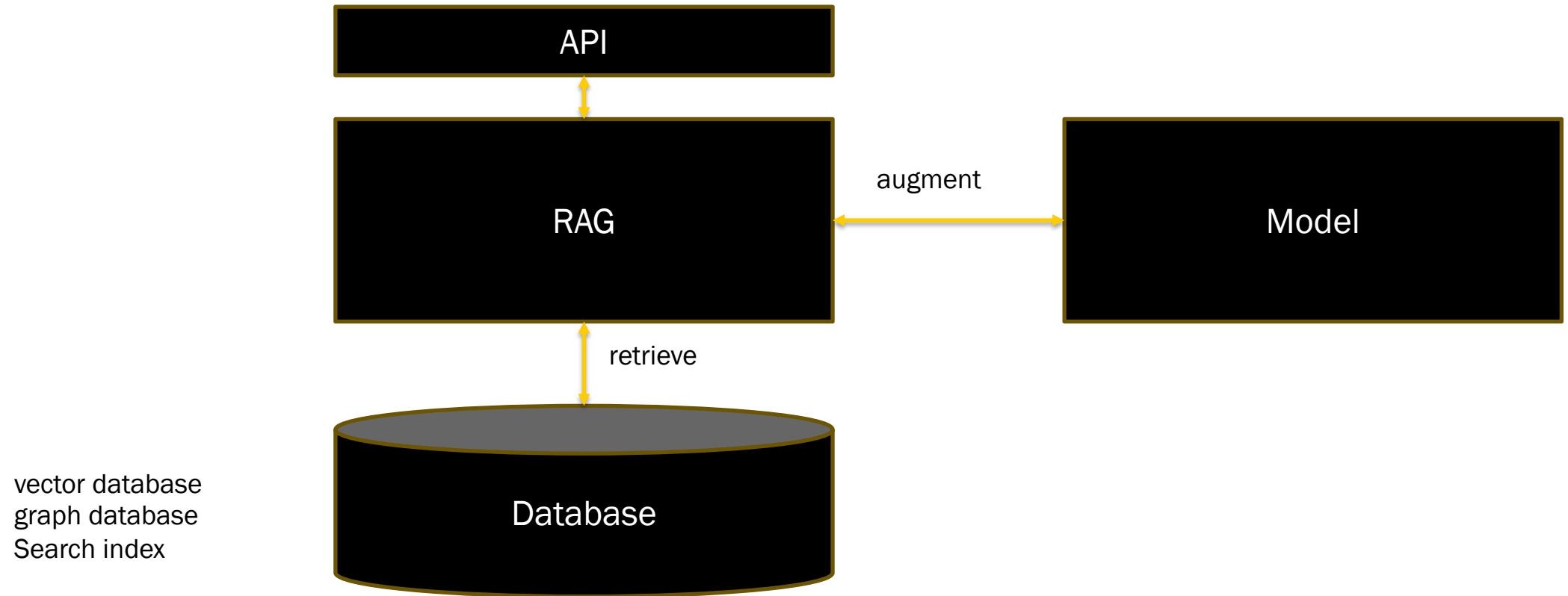
- <https://github.com/modelcontextprotocol/registry>



- <https://github.com/modelcontextprotocol/servers>
- <https://github.com/mcp>
- <https://mcp.so>

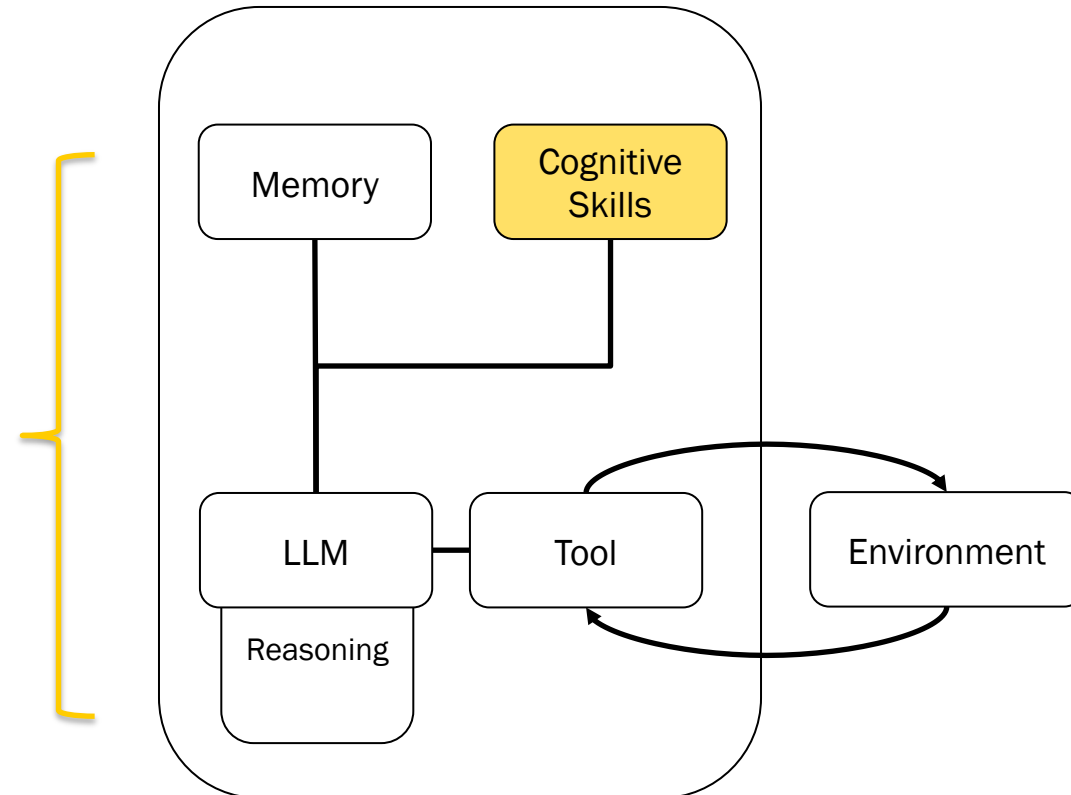
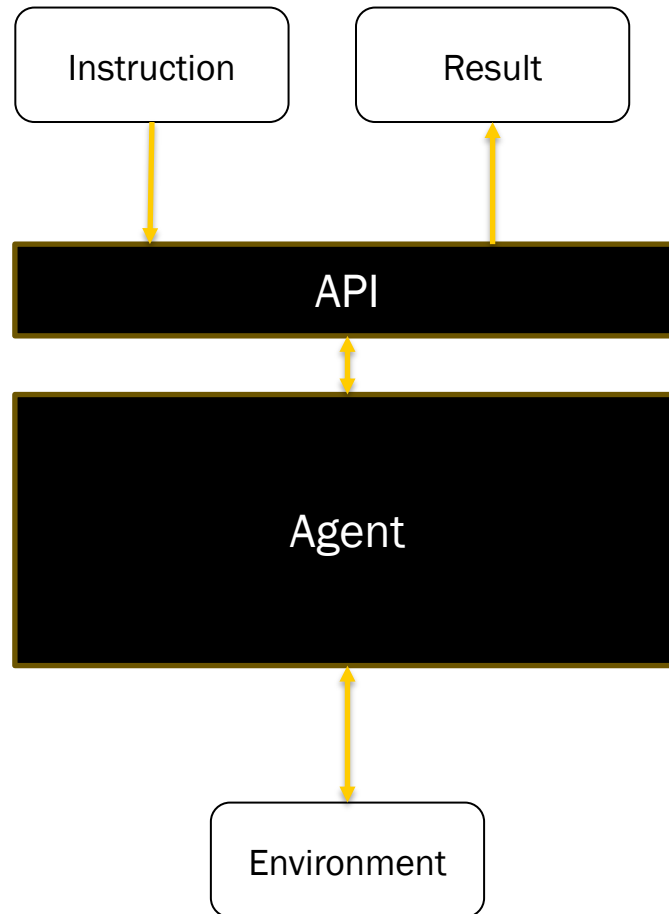
# AI Reference Architecture - RAG

## Retrieval Augmented Generation



# AI Reference Architecture - Agents

## Agents

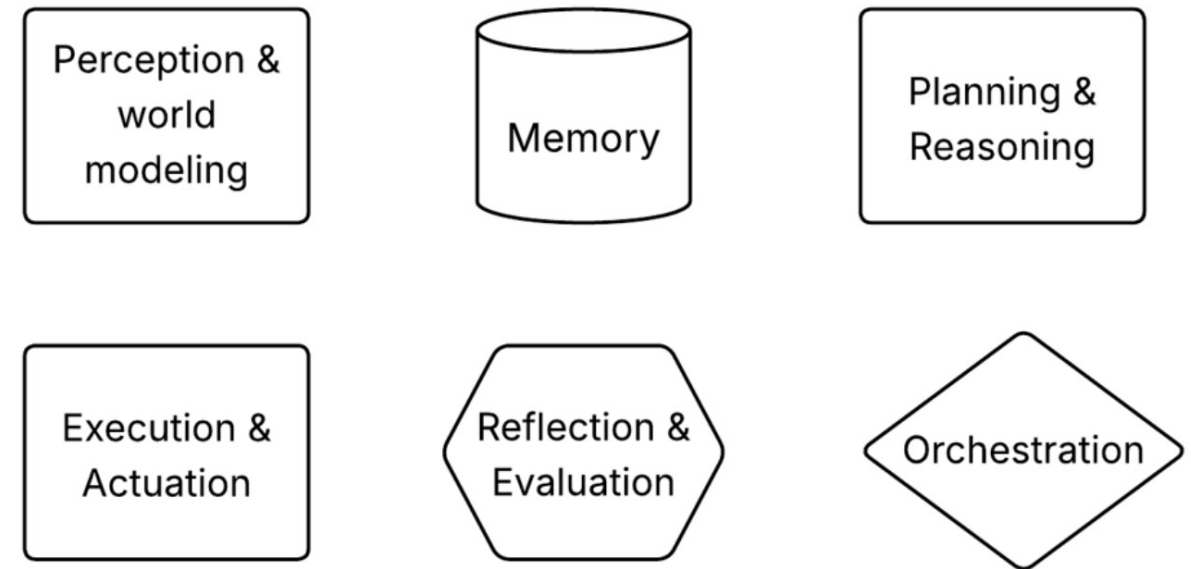


# AI Reference Architecture - Agents

## Agent Core Components

- **Perception and world modeling:** Ingests and structures external inputs (e.g., text, sensors, APIs) into internal representations.
- **Memory** (Short-Term, Long-Term, Episodic): Stores short- and long-term knowledge; retrieval/promotion rules connect past to present reasoning.
- **Planning, Reasoning, and Goal Decomposition:** Transforms goals into actionable steps, evaluates alternatives, and selects next actions.
- **Execution and Actuation:** Carries out actions via APIs or actuators, with monitoring and dynamic replanning.
- **Reflection and Evaluation:** Enables self-critique, verification, and refinement of actions and plans.
- **Communication, Orchestration, and Autonomy:** Coordinates task flow, retries, and timeouts, either centrally (e.g., LLM-based supervisor) or via decentralized protocols.

## Core Components of Agentic AI System

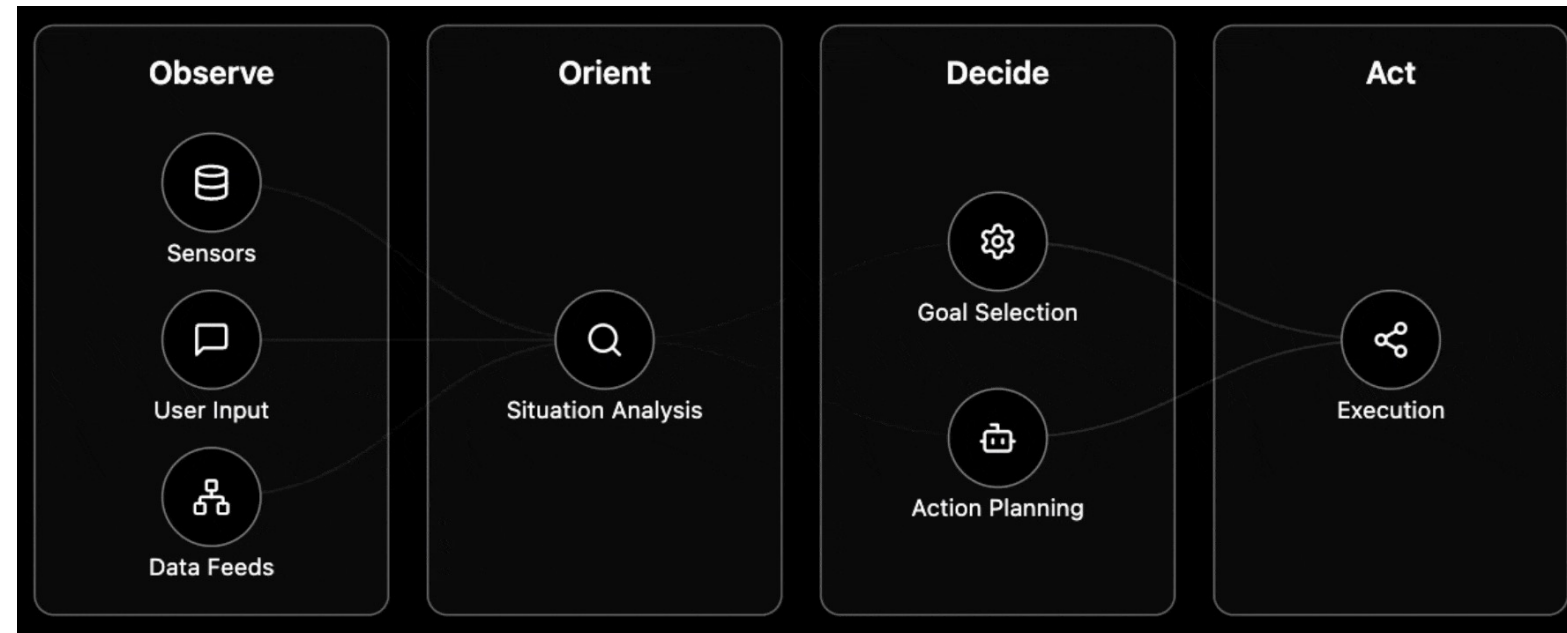


**Figure 4.** Core components of agentic AI system.

# AI Reference Architecture - Agents

## How Agents Works (OODA Loop)

1. Observe – It gathers data from sensors, user input, and feeds.
2. Orient – It analyzes the situation to understand what's happening.
3. Decide – It sets goals and plans the next move.
4. Act – It executes the task and takes action!

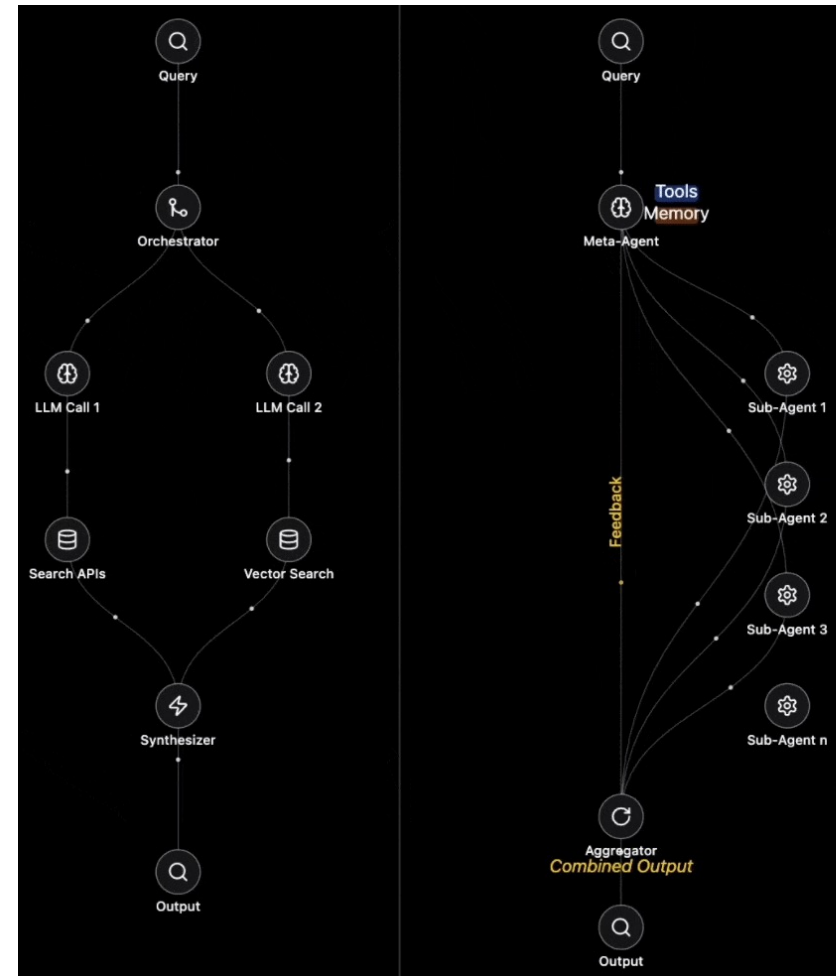


# AI Reference Architecture - Agents

## Orchestration: AI Workflow vs. Agentic AI

### AI Workflow

- **Linear Progression:** A structured, step-by-step pipeline starting from a query, passing through an orchestrator, and executing predefined tasks (e.g., LLM calls, Search APIs, Vector Search).
- **Centralized Processing:** A synthesizer consolidates results into a single output, ensuring predictability and consistency.
- **Deterministic Behavior:** Easier to manage, debug, and optimize for scenarios requiring transparency and control.



### AI Agent

- **Modular Design:** A meta-agent assigns tasks to specialized sub-agents, each handling a distinct aspect of the query.
- **Decentralized Execution:** Outputs from sub-agents are aggregated, with feedback loops enabling continuous refinement.
- **Enhanced Flexibility:** Supports parallel processing, iterative learning, and adaptation to complex, evolving problems.



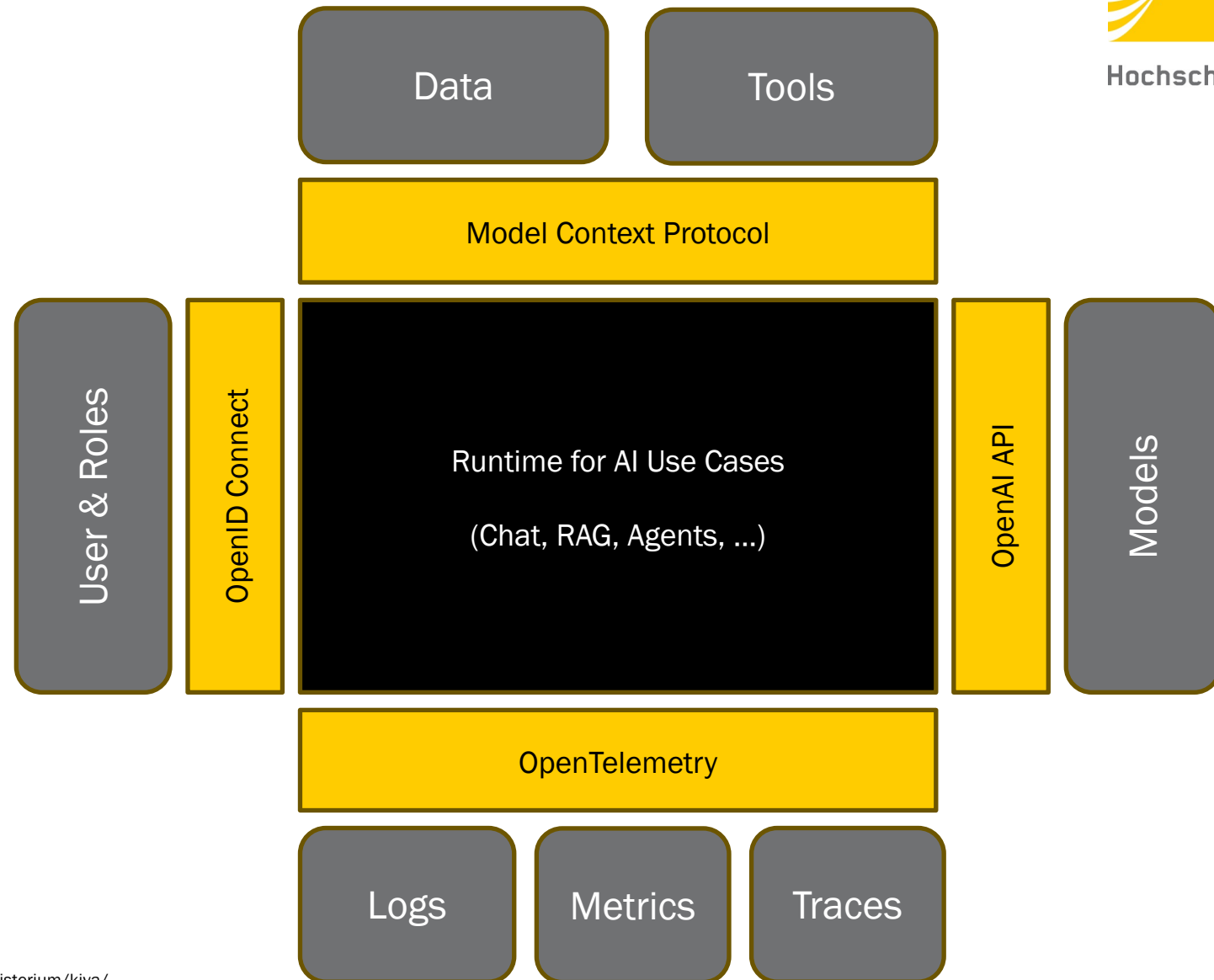
# 03.07.2

## AI Operating System



# AI Operating System

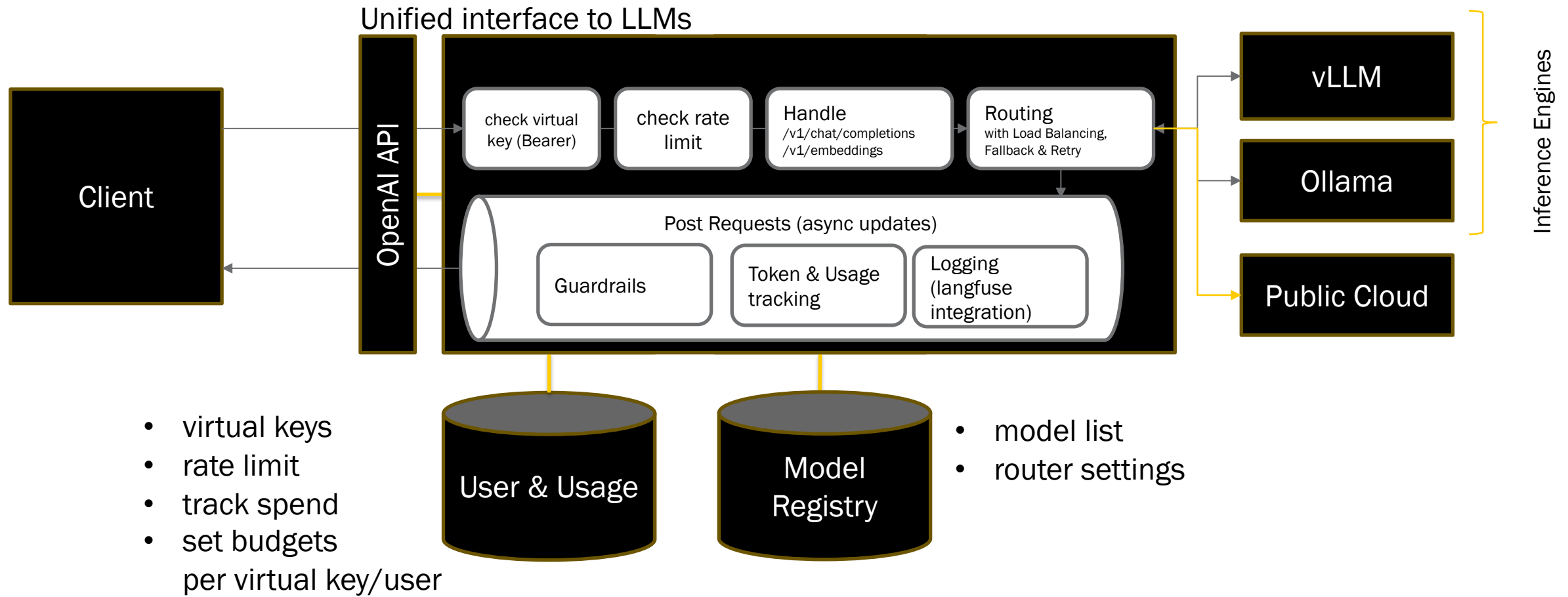
Offene Standards



Quelle: <https://baden-wuerttemberg.usercontent.opencode.de/innenministerium/kiva/>

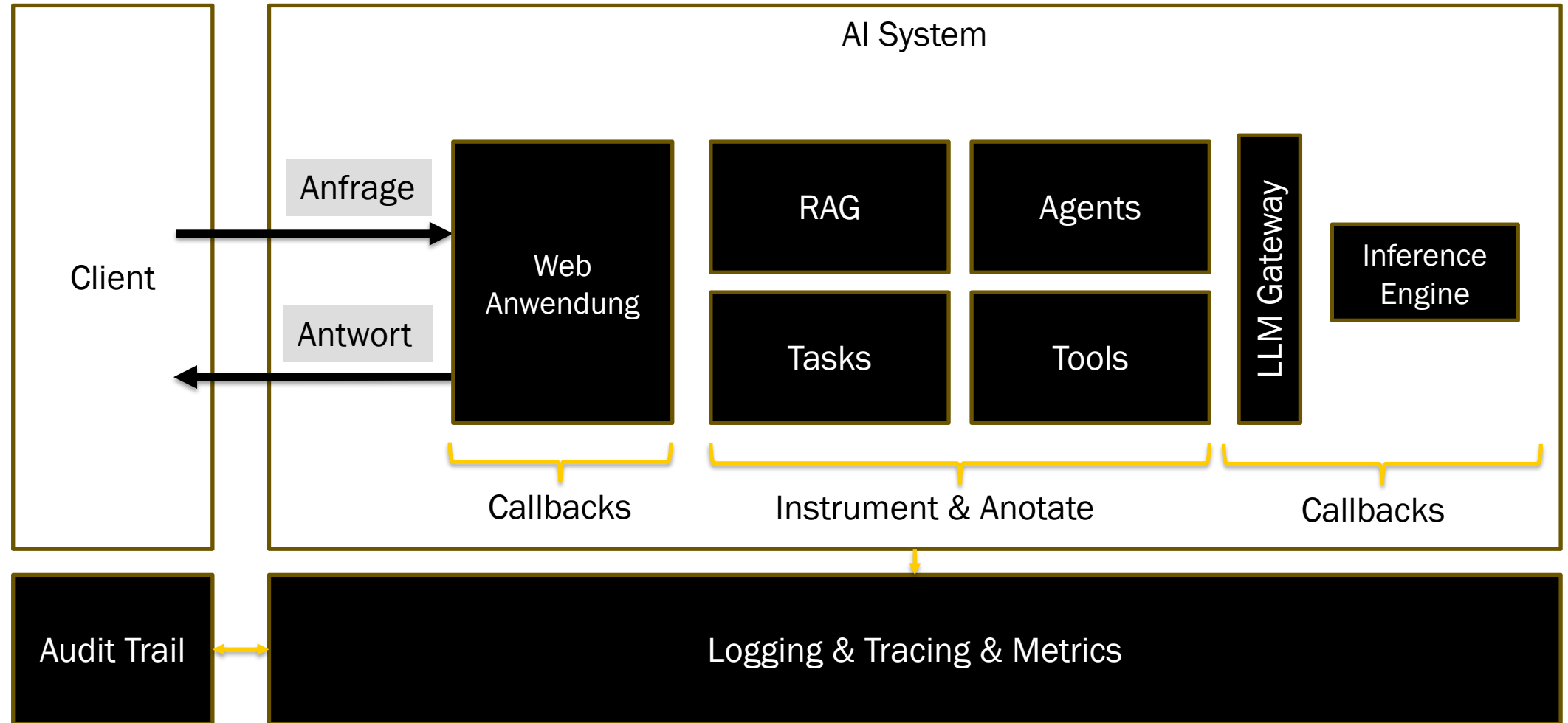
# AI Operating System

## Inference-Backend with LLM-Gateway: Runtime for model inference



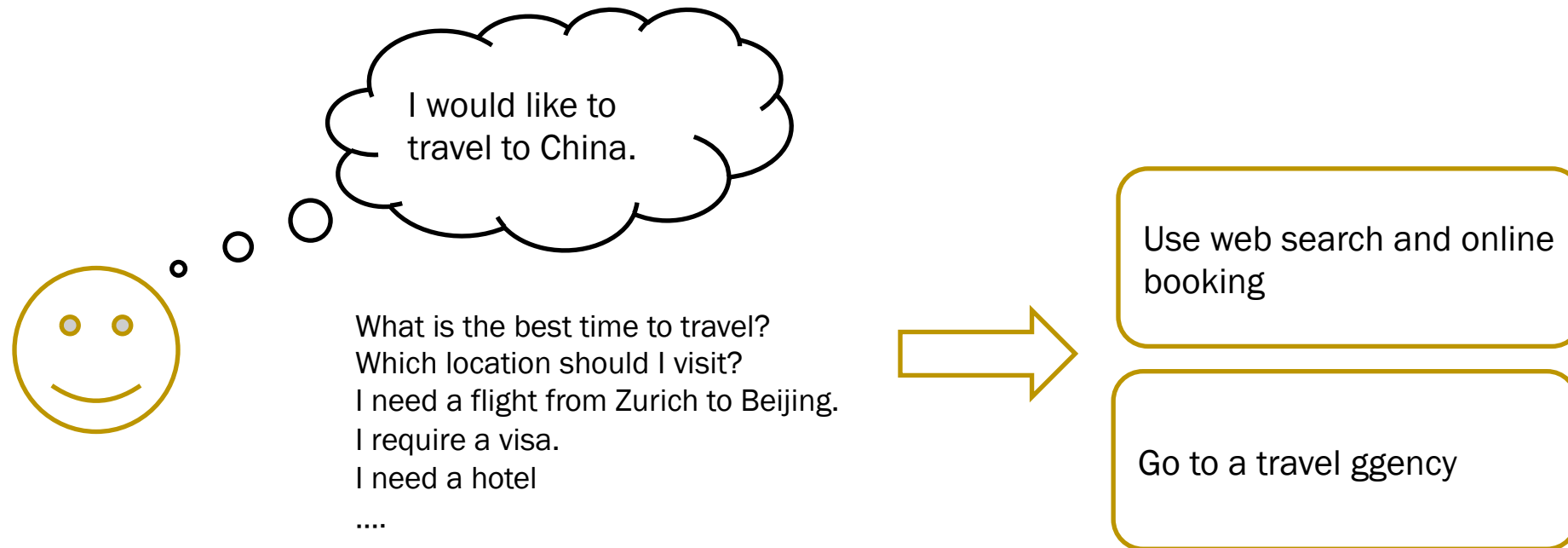
# AI Operating System

## End-to-End Observability



# 03.07.3 The Future is Agentic AI





This leads to 4 types of agent communication and orchestration:

1. A single Agent uses different tools
2. A workflow engine is orchestrating multiple agents, tools and tasks
3. Enterprise multi-agent orchestration (without workflow engine)
4. Decentralized cross-domain networks

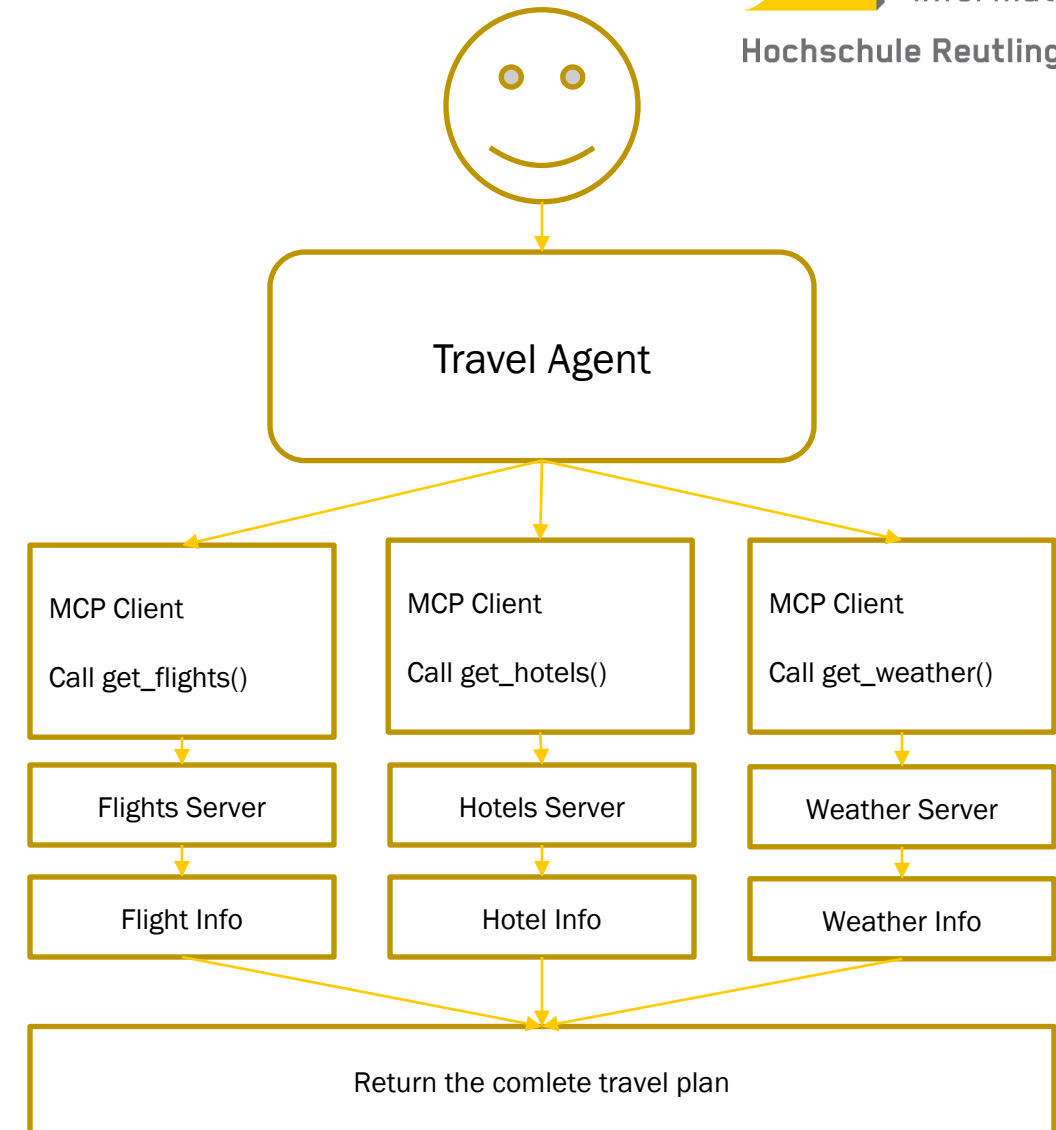
# Agentic AI: Single-agent tool orchestration

**Model Context Protocol (MCP)** protocol represents a centralized approach where a single agent coordinates all interactions with external services.

Travel agent serves as the coordinating agent with direct dependencies on all external services:

- MCP client directly invokes Flight Server, Hotel Server, and Weather Server through respective calls (`get_flights()`, `get_hotels()`, `get_weather()`).
- All external services are treated as tools that provide information but don't interact with each other.
- Information flow follows a strict star pattern with the MCP client at the center.
- The travel agent is responsible for aggregating all responses and generating the complete travel plan.

A Survey of AI Agent Protocols

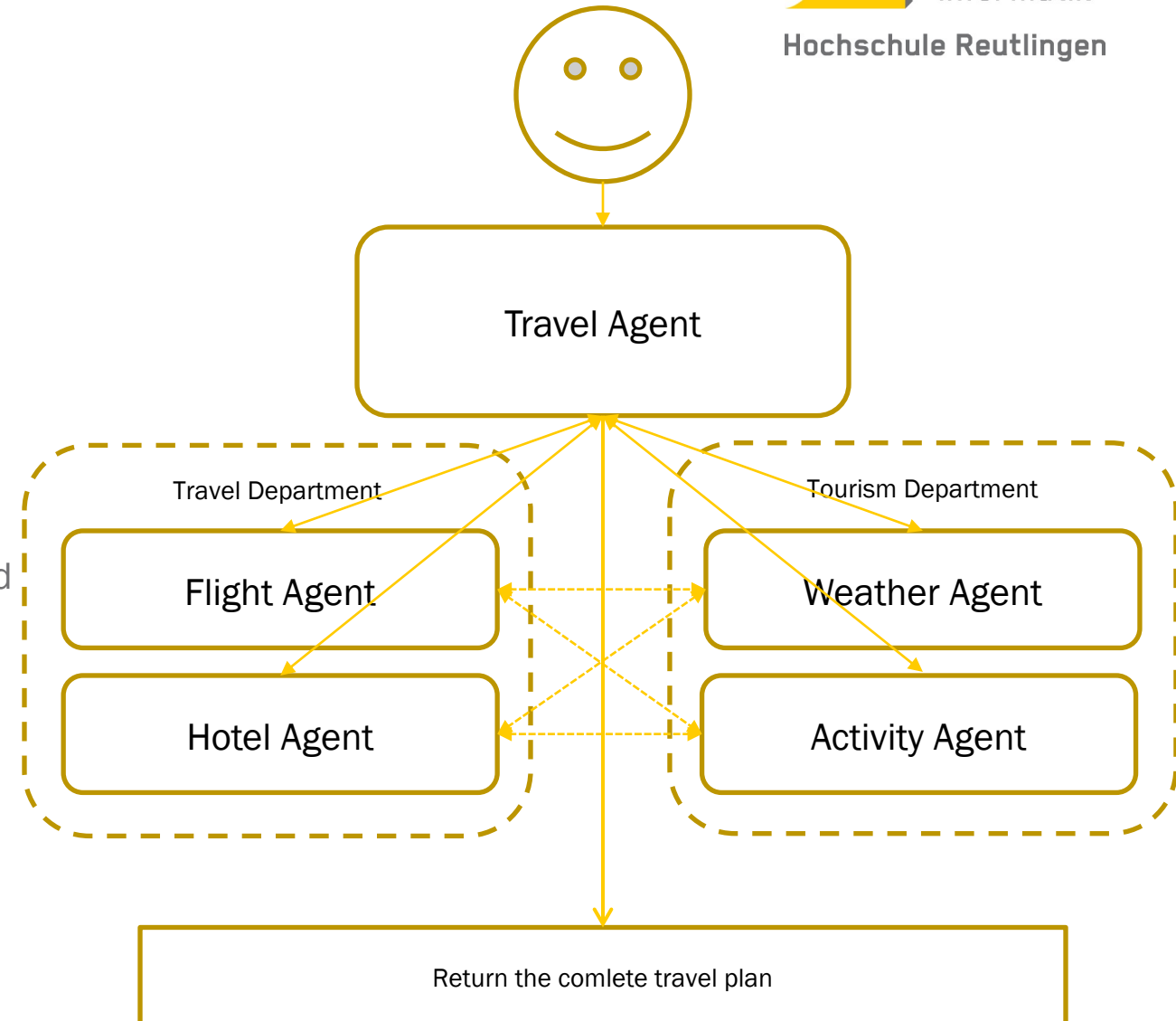


# Agentic AI: Enterprise multi-agent orchestration

**Agent-to-Agent (A2A)** protocol enables direct communication between different agents for complex tasks.

The A2A implementation distributes intelligence across multiple specialized agents:

- Agents are organized into logical departments (Transportation, Accommodation & Activities).
- Each agent has explicit dependencies: the Flight Agent and Activity Agent depend on the Weather Agent for environmental data.
- Agents communicate directly with each other, without requiring central coordination for every interaction.
- The A2A Travel Planner functions as a non-central coordinator that primarily collects final results.



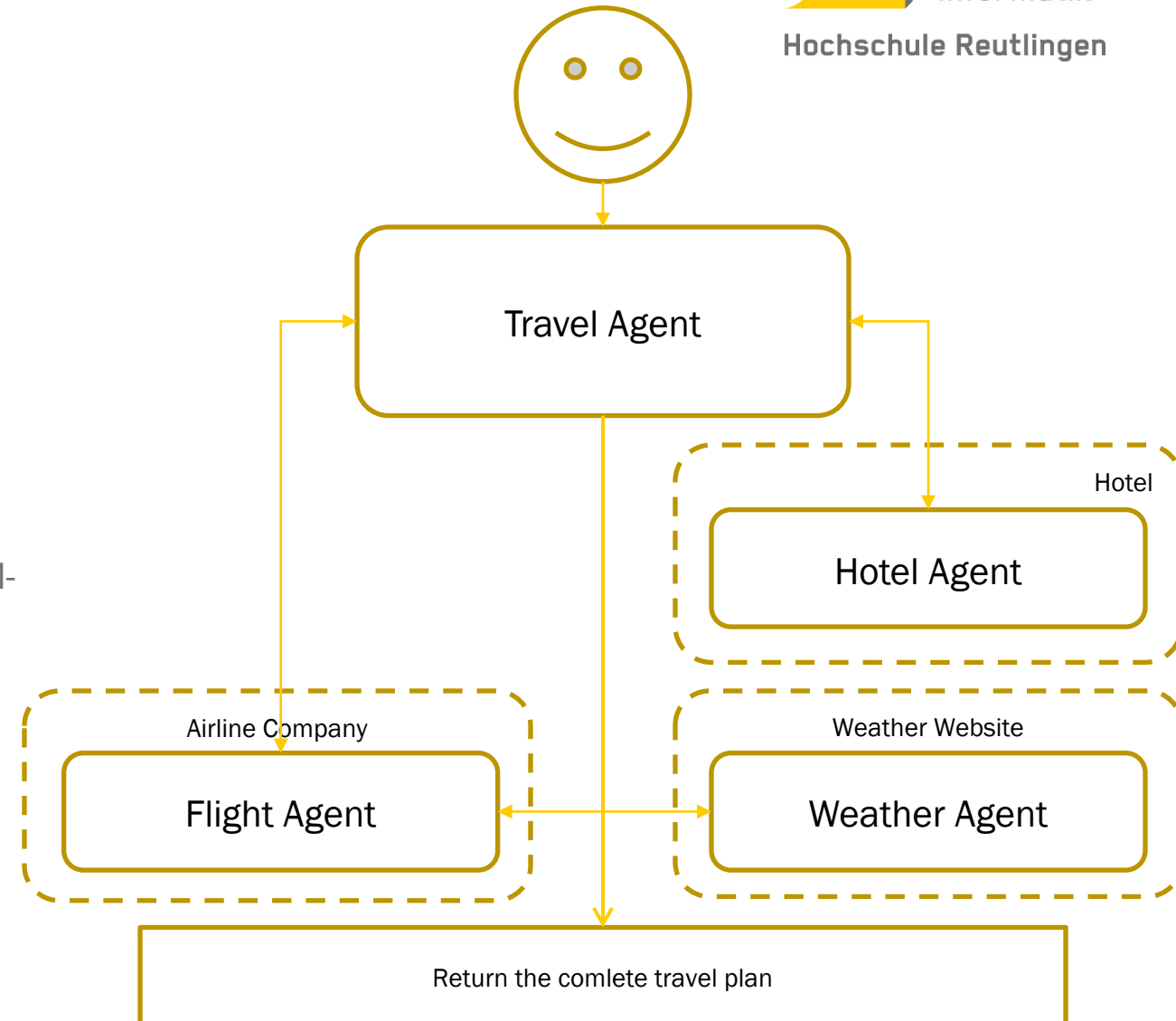


# Agentic AI: Decentralized cross-domain networks

**Agent Network Protocol (ANP)** extends collaboration through standardized cross-domain interactions.

ANP enables negotiations between agents in different organizational domains:

- Distinct organizational boundaries separate the Airline Company, Hotel, and Weather Website.
- Cross-domain collaboration occurs through formal protocol-based requests and responses.
- The Flight Agent negotiates with the Weather Agent across domain boundaries.
- The Travel Planner coordinates the overall process but doesn't mediate every interaction.

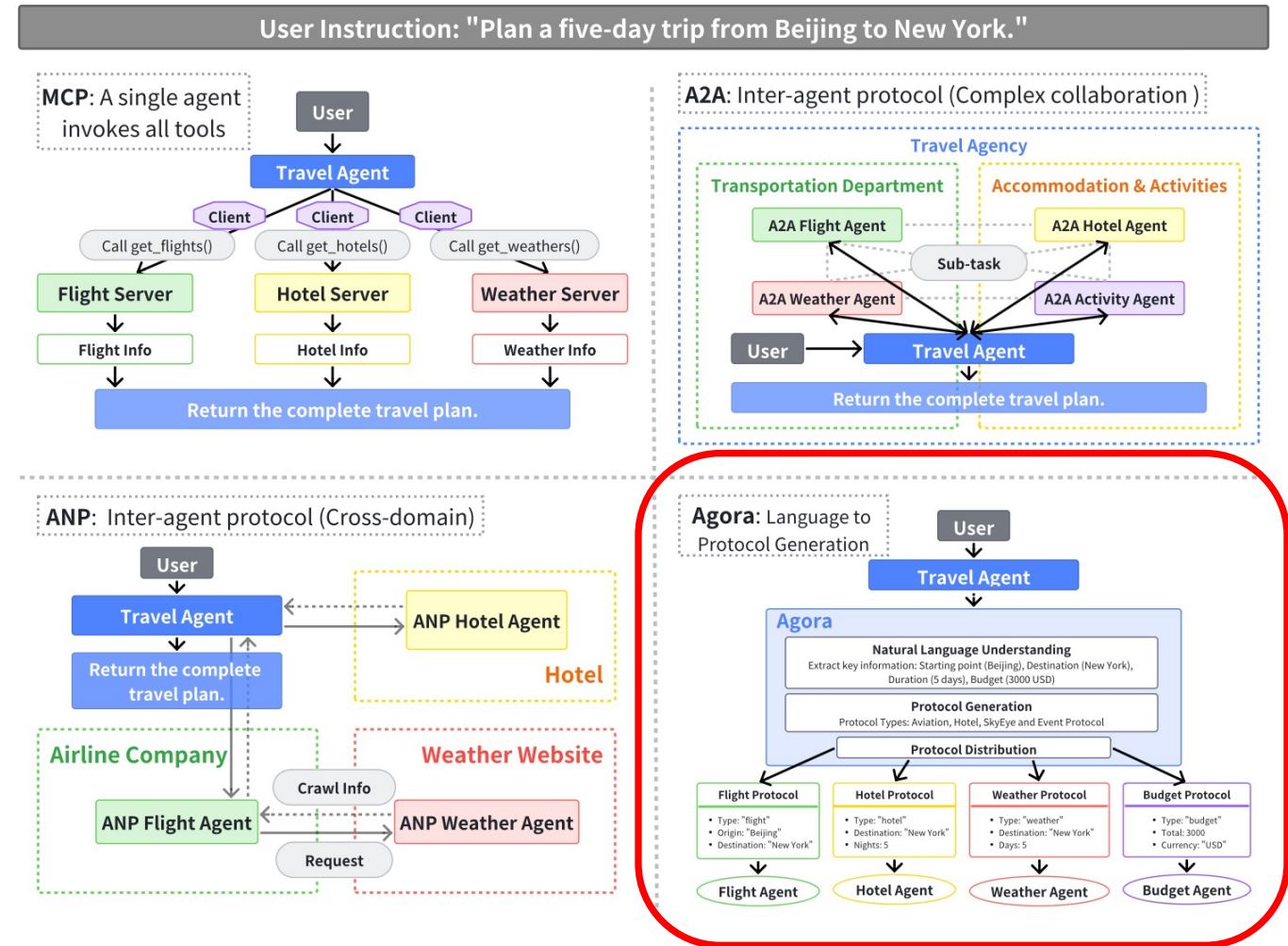


# Agentic AI: Agora Language to Protocol Generation

The **Agora** protocol represents the most user-centric approach, converting natural language requests directly into standardized protocols.

Agora introduces several distinctive layers:

- The process begins with natural language understanding, parsing the user's request into structured components (origin, destination, duration, budget).
- The protocol generation layer transforms these components into formalized protocols for different service types.
- Protocol distribution dispatches the appropriate protocols to specialized agents
- Each agent responds to its specific protocol rather than to free-form requests.



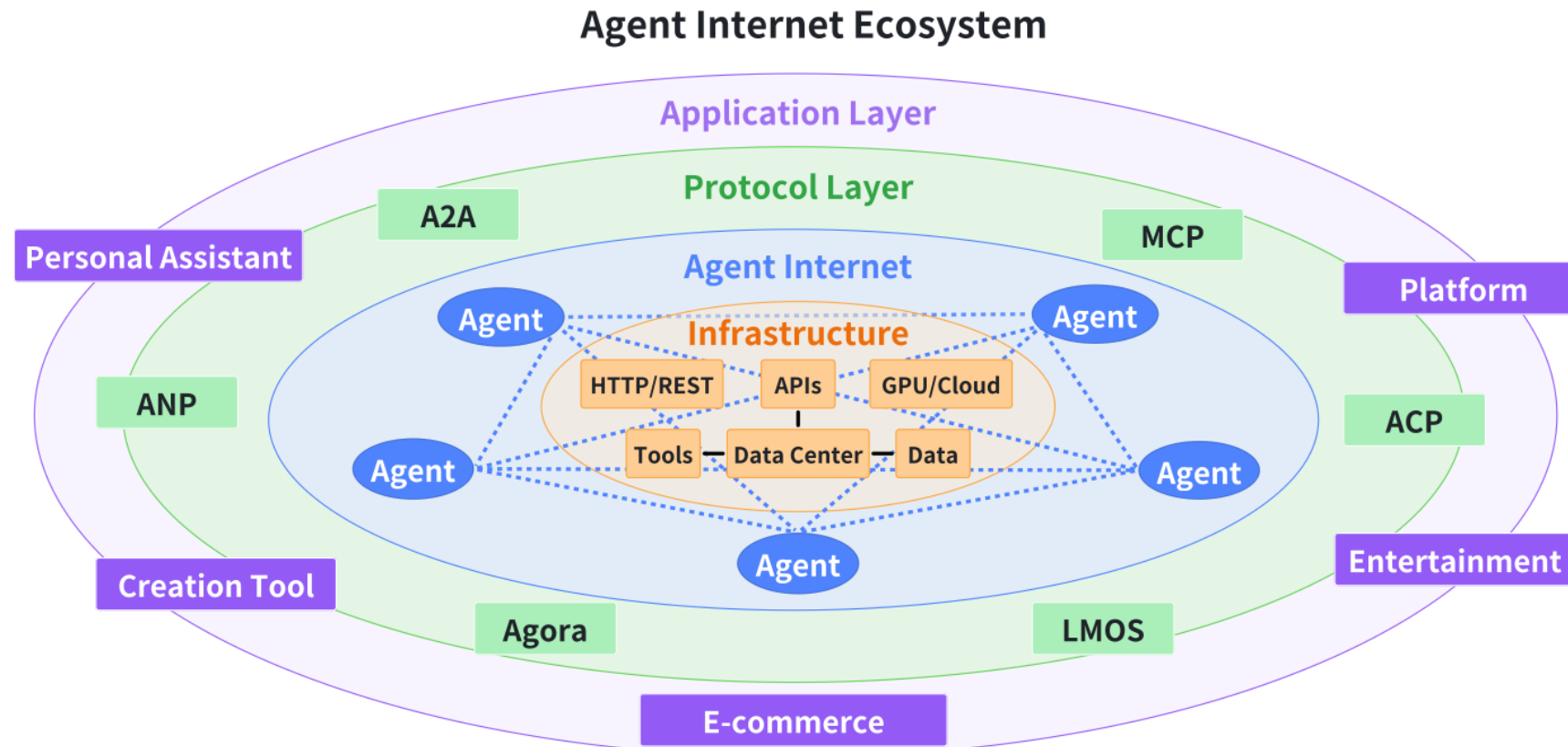
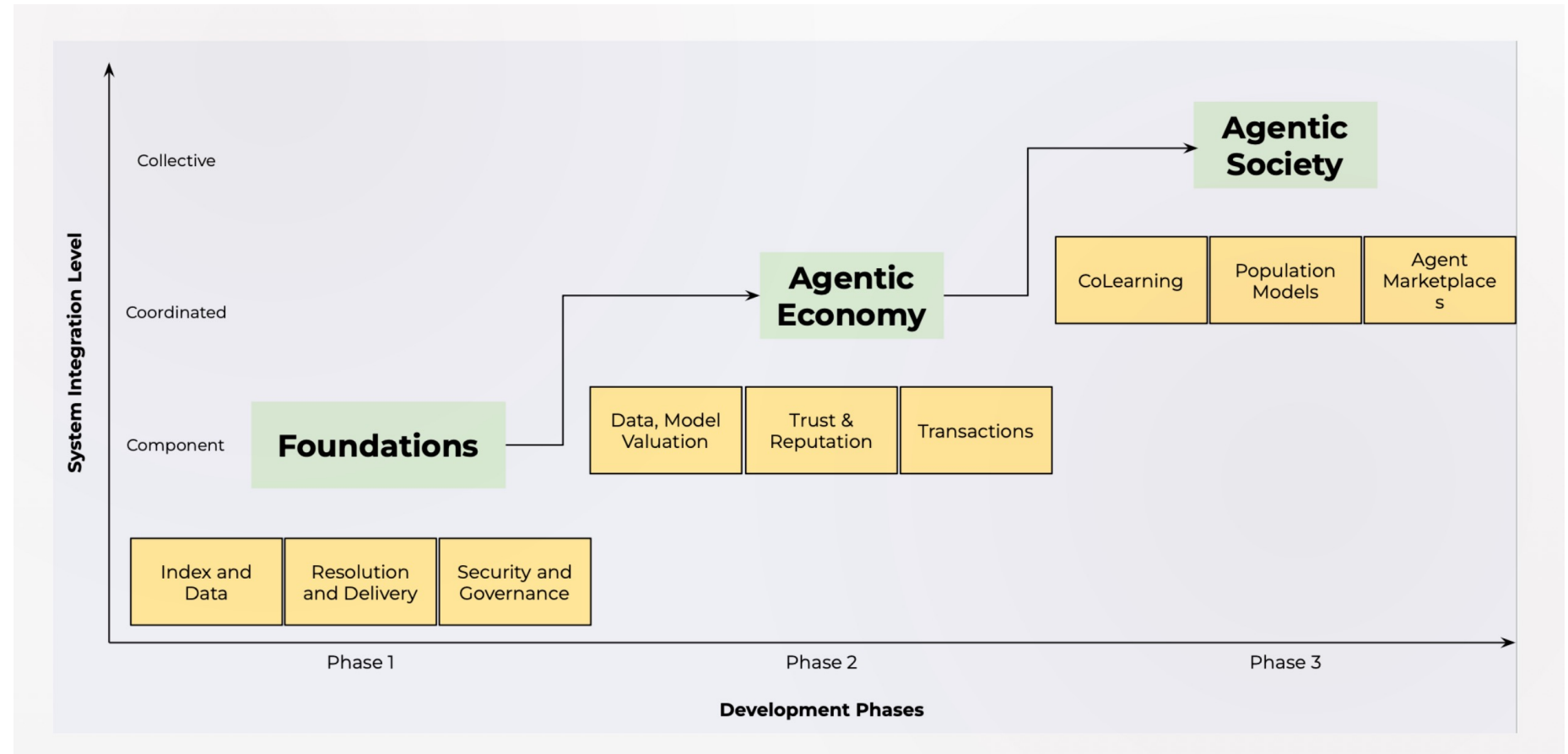


Figure 1: A layered architecture of the Agent Internet Ecosystem.

# NANDA: The Internet of AI Agents

Imagine billions of specialized AI agents collaborating across a decentralized architecture.

Each performs discrete functions while communicating seamlessly, navigating autonomously, socializing, learning, earning and transacting on our behalf.



Source: <https://nanda.media.mit.edu>

# Literature



# MCP Server



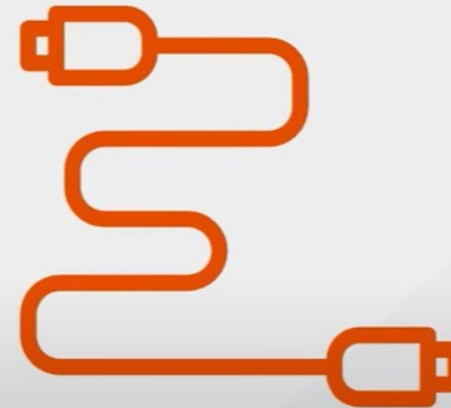
# From Function Calling to MCP

Function Calling was a big step for agentic ai, but not standardized!

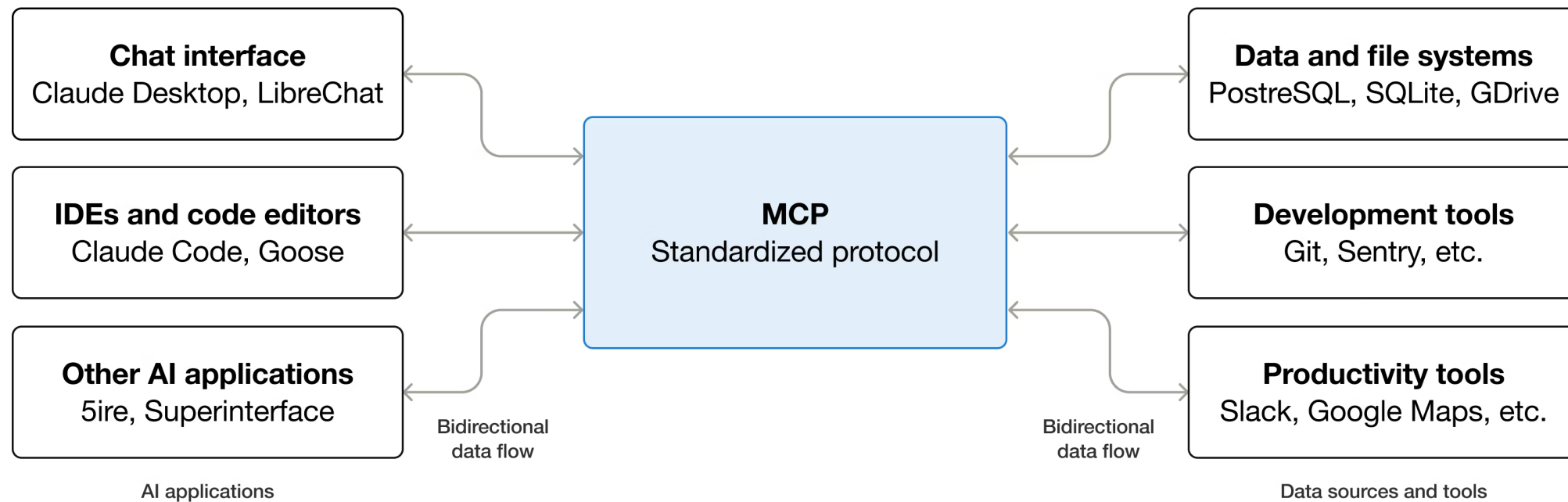
## MCP (Model Context Protocol)

- is an open-source standard for connecting AI applications to external systems.
- Using MCP, AI applications like Claude or ChatGPT can connect to:
  - data sources (e.g. local files, databases),
  - tools (e.g. search engines, calculators) and
  - workflows (e.g. specialized prompts)

## Think of MCP like USB-C

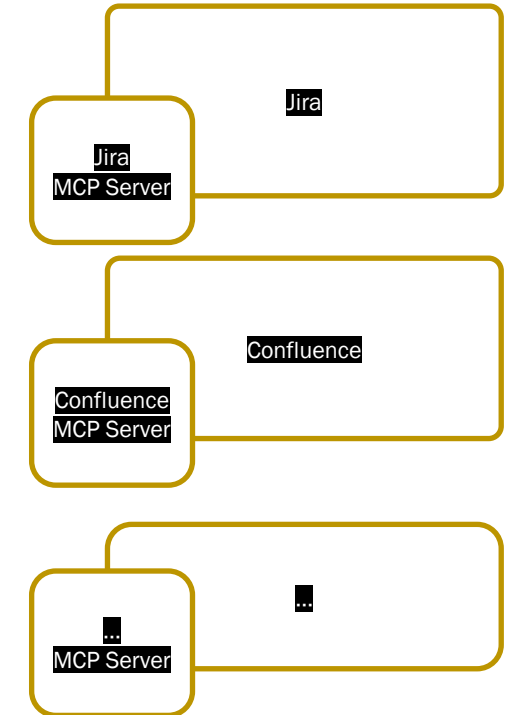
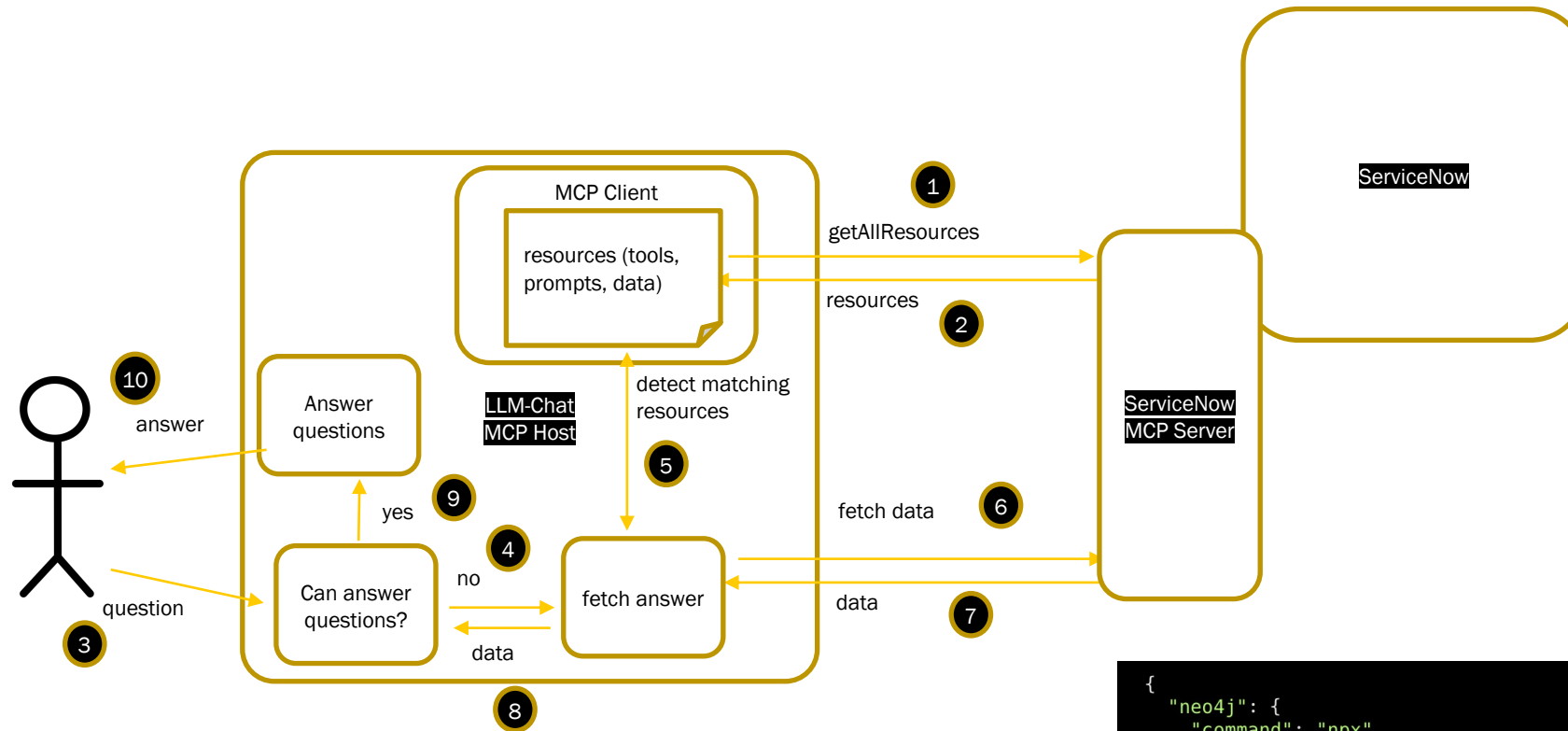


**One universal connector for all tools. Once it speaks MCP, it just works.**





# Tools: Model Context Protocol



```
{
  "neo4j": {
    "command": "npx",
    "args": [
      "@alanse/mcp-neo4j-server"
    ],
    "env": {
      "NEO4J_URI": "bolt://localhost:7687",
      "NEO4J_DATABASE": "neo4j",
      "NEO4J_PASSWORD": "admin",
      "NEO4J_USERNAME": "admin"
    },
    "working_directory": null
  }
}
```

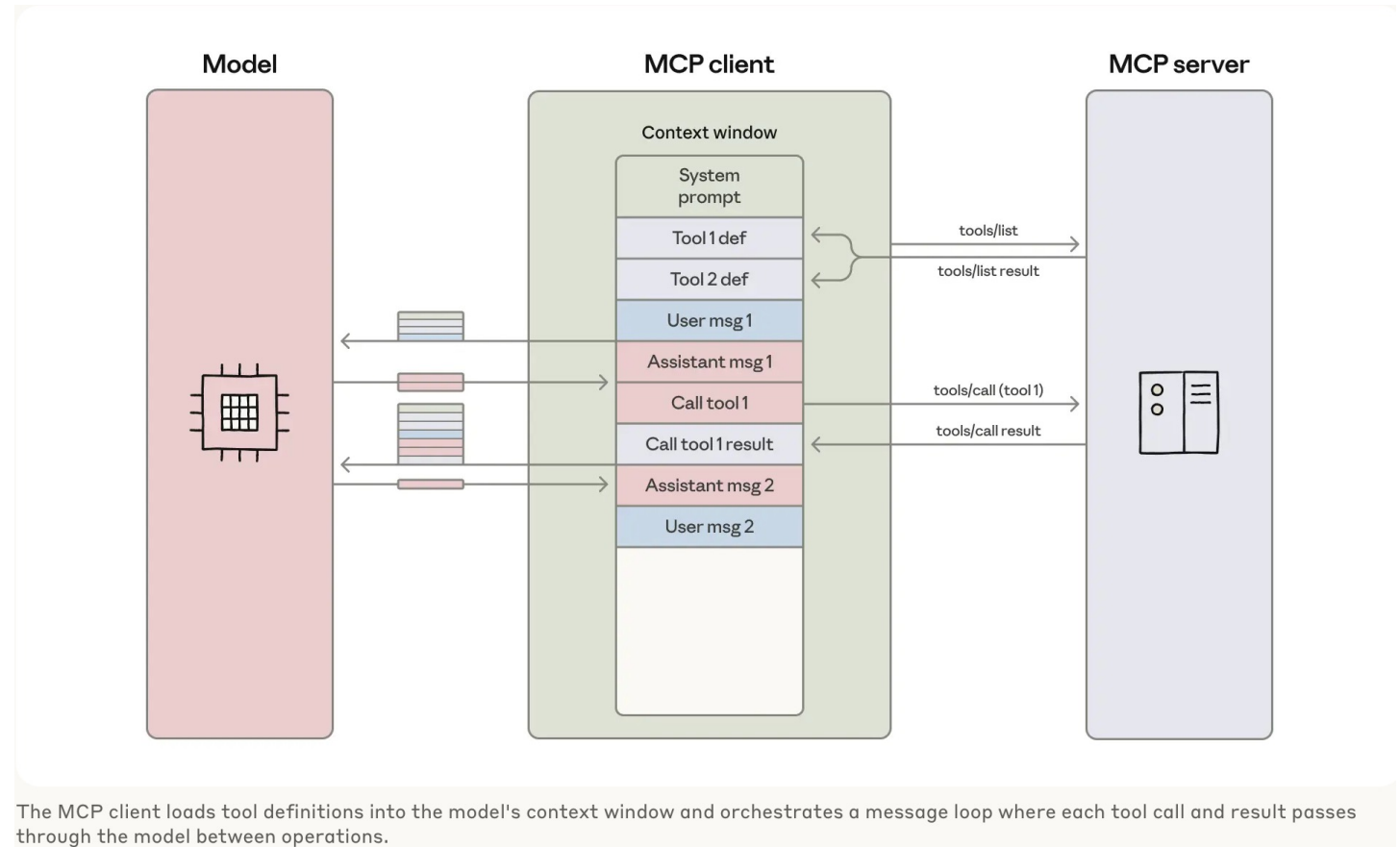
# Tools: Model Context Protocol

## Herausforderungen

Problem: Massive Token Consumption and Context Window Size.

Solution: Code execution with MCP

Presenting tools as code on a filesystem allows models to read tool definitions on-demand, rather than reading them all up-front.



# Tools: From Claude Agent Skills to Learning a Toolbox

Agent Skills are modular capabilities that extend Claude's functionality.

## Each Skill packages:

- instructions,
- metadata, and
- optional resources (scripts, templates)

that Claude uses automatically when relevant.

```
pdf-skill/  
├── SKILL.md (main instructions)  
├── FORMS.md (form-filling guide)  
├── REFERENCE.md (detailed API reference)  
└── scripts/  
    └── fill_form.py (utility script)
```

→ Skills are folders of reusable instructions, scripts, and resources for models to improve performance on specialized tasks.

# Tools: From Claude Agent Skills to Learning a Toolbox

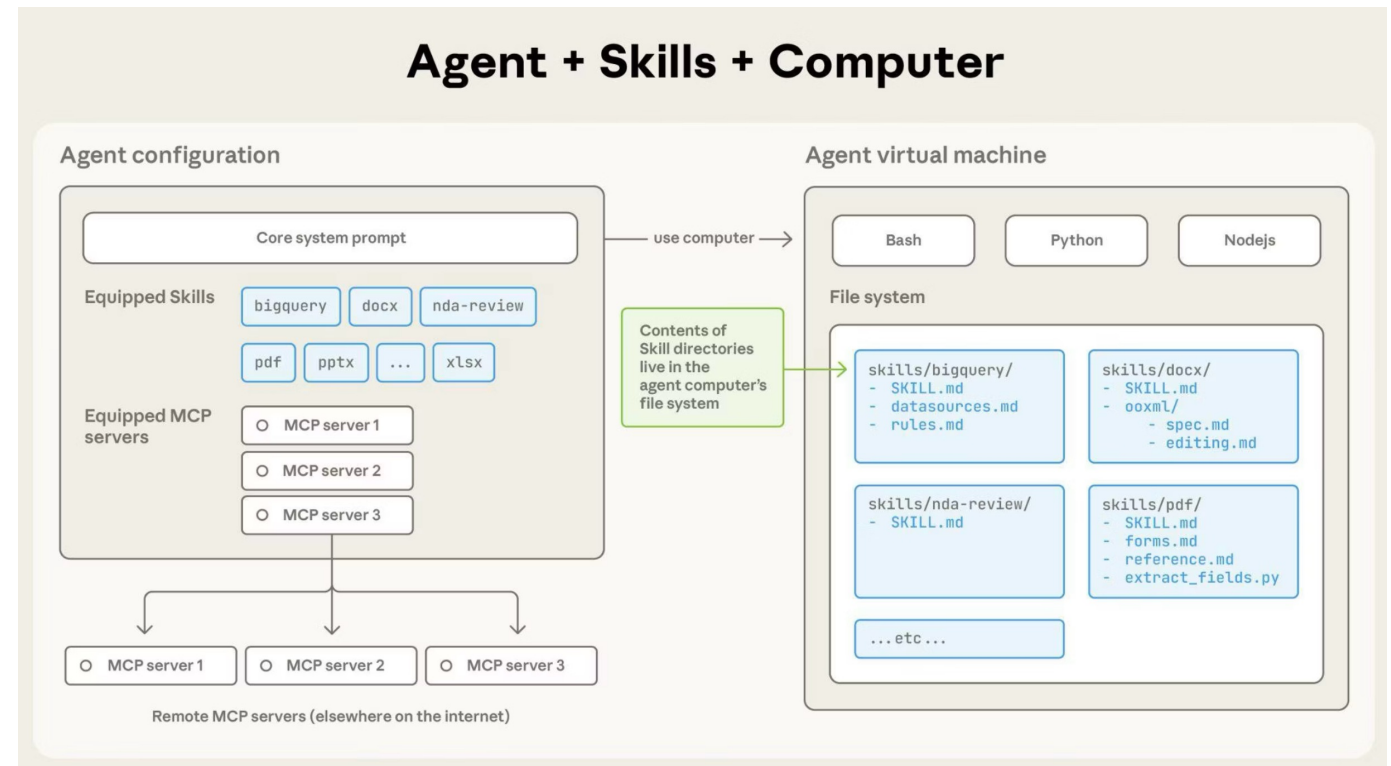
Runtime:

Agent Skills need a Runtime for execution:

Next:

What if agents can also persist their own code as reusable functions. Once an agent develops working code for a task, it can save that implementation for future use. Then Adding a SKILL.md file to these saved functions creates a structured skill that models can reference and use. Over time, this allows your agent to build a toolbox of higher-level capabilities, evolving the scaffolding that it needs to work most effectively.

Source: <https://docs.claude.com/en/docs/agents-and-tools/agent-skills/overview>



Running agent-generated code requires a secure execution environment with appropriate sandboxing, resource limits, and monitoring. These infrastructure requirements add operational overhead and security considerations that direct tool calls avoid. The benefits of code execution - reduced token costs, lower latency, and improved tool composition —should be weighed against these implementation costs.

- Filesystem isolation, which ensures that Claude can only access or modify specific directories. This is particularly important in preventing a prompt-injected Claude from modifying sensitive system files.
- Network isolation, which ensures that Claude can only connect to approved servers. This prevents a prompt-injected Claude from leaking sensitive information or downloading malware.

<https://github.com/anthropic-experimental/sandbox-runtime>

Source: <https://www.anthropic.com/engineering/claude-code-sandboxing>

